

LAPORAN PROJECT KELOMPOK D



Mata Kuliah Algoritma Pemrograman
Kelas Statistika C
Nama Dosen Lutfiah Maharani Siniwi, S.Stat., M.Stat.

Anggota Kelompok:

Meyluna Prachi Sri Rumondang Purba	K1D025091
Khalisa Irfani	K1D025093
Nadien Dwi Anggraeni	K1D025098
Safira Aulia	K1D025103

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

PURWOKERTO

2026

DAFTAR ISI

DAFTAR ISI BAB I	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Project	1
1.3 Ruang Lingkup Project	2
BAB II ANALISIS MASALAH	3
2.1 Analisis Program 1	3
2.2 Analisis Program 2	4
2.3 Analisis Program 3	5
2.4 Analisis Program 4	7
2.5 Analisis Program 5	8
2.6 Analisis Program 6	10
2.7 Pembagian Tugas Kelompok	10
BAB III PSEUDOCODE DAN FLOWCHART	12
3.1 Pseudocode dan Flowchart program 1	12
3.2 Pseudocode dan Flowchart program 2	13
3.3 Pseudocode dan Flowchart Program 3	14
3.4 Pseudocode dan Flowchart program 4	15
3.5 Pseudocode dan Flowchart program 5	17
3.6 Pseudocode dan Flowchart program 6	19
BAB IV IMPLEMENTASI PROGRAM	21
4.1 Source code Python dan R Program 1	21
4.2 Source code Python dan R Program 2	22
4.3 Source code Python dan R Program 3	25
4.4 Source code Python dan R Program 4	26
4.5 Source code Python dan R Program 5	29
4.6 Source code Python dan R Program 6	31
BAB V	34

BAB V PENJELASAN LINE-BY-LINE CODE	34
5.1 Tabel penjelasan setiap baris kode Python dan R Program 1	34
5.2 Tabel penjelasan setiap baris kode Python dan R Program 2	37
5.3 Tabel penjelasan setiap baris kode Python dan R Program 3	39
5.4 Tabel penjelasan setiap baris kode Python dan R Program 4	41
5.5 Tabel penjelasan setiap baris kode Python dan R Program 5	49
5.6 Tabel penjelasan setiap baris kode Python dan R Program 6	53
BAB VI PENGUJIAN PROGRAM	56
6.1 Pengujian Program 1	56
6.3 Pengujian Program 3	57
6.4 Pengujian Program 4	59
6.5 Pengujian Program 5	61
6.6 Pengujian Program 6	64
BAB VII PENUTUP	66
7.1 Kesimpulan	66
7.2 Kendala	66
7.3 Saran.....	67
LAMPIRAN	68

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong penggunaan komputer untuk menyelesaikan berbagai permasalahan secara efektif dan efisien. Dalam pengembangan perangkat lunak, algoritma berperan sebagai langkah-langkah logis yang digunakan untuk menyelesaikan suatu masalah, sedangkan pemrograman merupakan implementasi algoritma tersebut ke dalam bahasa yang dapat dipahami oleh komputer. Oleh karena itu, pemahaman terhadap konsep algoritma dan pemrograman menjadi keterampilan dasar yang harus dimiliki oleh mahasiswa di bidang komputasi.

Mata kuliah Algoritma dan Pemrograman memberikan pemahaman mengenai proses analisis masalah, perancangan algoritma, pembuatan flowchart, penyusunan pseudocode, serta implementasi program menggunakan bahasa pemrograman. Melalui project ini, mahasiswa dituntut untuk menerapkan konsep-konsep tersebut pada berbagai studi kasus yang mencakup pengolahan teks, manipulasi string, perhitungan matematis, pengolahan data identitas, klasifikasi data berdasarkan jarak, serta perhitungan interval kepercayaan statistik.

Project ini dikerjakan menggunakan dua bahasa pemrograman, yaitu Python dan R. Penggunaan kedua bahasa tersebut bertujuan agar mahasiswa dapat memahami perbedaan sintaks dan karakteristik masing-masing bahasa tanpa mengubah logika algoritma yang digunakan. Selain itu, project ini juga melatih kemampuan mahasiswa dalam melakukan validasi input, menangani kondisi khusus, menguji program secara sistematis, serta bekerja sama dalam tim menggunakan repository GitHub.

1.2 Tujuan Project

Tujuan pelaksanaan project Algoritma dan Pemrograman ini adalah sebagai berikut:

1. Menganalisis permasalahan komputasional dengan menentukan input, proses, output, serta batasan masalah.
2. Menyusun algoritma dalam bentuk pseudocode dan flowchart yang sistematis.
3. Mengimplementasikan algoritma menggunakan bahasa pemrograman Python dan R.
4. Membandingkan struktur dan logika program pada kedua bahasa pemrograman tersebut.
5. Menerapkan konsep variabel, tipe data, operator, string, percabangan, fungsi, serta operasi matematika dalam penyelesaian masalah.
6. Melakukan validasi input dan penanganan kondisi khusus pada program.
7. Melakukan pengujian program untuk memastikan hasil yang diperoleh sesuai dengan kebutuhan dan ketentuan soal.

1.3 Ruang Lingkup Project

Ruang lingkup project ini meliputi perancangan dan implementasi enam program yang diberikan pada tugas project mata kuliah Algoritma dan Pemrograman, yaitu:

1. Program analisis teks untuk menghitung jumlah kata dan jumlah kalimat.
2. Program penggunaan string untuk menampilkan kalimat sesuai format yang ditentukan.
3. Program perhitungan akar-akar persamaan kuadrat.
4. Program pengolahan NIP ASN untuk menampilkan tanggal lahir berdasarkan NIP.
5. Program klasifikasi cluster tiga dimensi berdasarkan jarak Euclidean.
6. Program perhitungan interval konfidensi proporsi berdasarkan nilai proporsi sampel dan tingkat kepercayaan.

Setiap program akan dibahas melalui tahapan analisis masalah, penyusunan pseudocode, pembuatan flowchart, implementasi kode Python dan R, penjelasan kode program, serta pengujian program menggunakan berbagai skenario input. Dengan demikian, project ini mencakup seluruh proses pengembangan program secara sistematis.

BAB II

ANALISIS MASALAH

2.1 Analisis Program 1

a. Deskripsi Soal

Program dibuat untuk menghitung seluruh jumlah kata dan jumlah kalimat dalam sebuah teks. Pengguna memasukkan sebuah teks yang terdiri dari satu atau lebih kalimat. Program kemudian menghitung banyak kata dan banyak kalimat berdasarkan tanda titik (.) sebagai penanda akhir kalimat.

b. Tabel Input-Proses-Output :

Komponen	Keterangan
Input	Sebuah teks dari pengguna
Proses	Menghitung jumlah kata menggunakan pemisahan berdasarkan spasi dan menghitung jumlah kalimat berdasarkan jumlah tanda titik
Output	Jumlah kata dan jumlah kalimat

c. Batasan Masalah :

- Tanda titik (.) hanya digunakan untuk mengakhiri kalimat.
- Tidak ada singkatan seperti "S.H.", "Dr.", "dll."
- Kata dipisahkan oleh spasi.
- Input berupa teks biasa

d. Kondisi Khusus :

1. Input teks kosong : Program menghasilkan jumlah kata=0 dan jumlah kalimat = 0.
2. Teks tanpa tanda titik (.) : Jumlah kata tetap dihitung, sedangkan jumlah kalimat = 0.
3. Teks terdiri dari beberapa kalimat : Setiap tanda titik (.) dihitung sebagai akhir satu kalimat sesuai asumsi soal.

2.2 Analisis Program 2

a. Deskripsi Soal

Program kedua ini menunjukkan bagaimana Bahasa pemrograman menangani data bertipe string. Pada kasus ini program 2 meminta untuk membuat empat variabel string yang membuat kombinasi tanda petik tunggal, tanda petik ganda, dan karakter khusus tertentu yang bertujuan untuk membuat empat objek string (K1, K2, K3, dan K4) dalam Bahasa Python dan R sehingga saat *diprint* menghasilkan kalimat yang sama persis dengan soal yang diberikan. Tantangan utama dari program ini terletak pada penulisan sintaks yang tepat sehingga isi teks dapat ditampilkan sama persis dengan kalimat yang diberikan tanpa menimbulkan kesalahan saat program di jalankan.

b. Tabel-Input-Proses-Output

Komponen	Keterangan
Input	Membuat input objek string K1, K2, K3, dan K4 sesuai ketentuan soal.
Proses	Tidak ada proses.
Output	Print isi K1, K2, K3, dan K4.

b. Batasan Masalah

2. Kalimat yang disimpan dalam K1, K2, K3, K4 harus sama persis dengan yang diminta soal.
3. Program harus memastikan seluruh string ditulis dengan pasangan tanda petik yang lengkap agar tidak terjadi kesalahan sintaks (Syntax Error), karakter khusus seperti garis miring (/), titik (.), dan tanda kutip harus tetap ditampilkan sesuai isi teks asli.
4. Program hanya berfungsi untuk menyimpan dan menampilkan string, tidak ada perhitungan angka.

c. Kondisi Khusus :

1. Kesalahan dalam penulisan tanda kutip dapat membuat program tidak bisa dijalankan.

2. Beberapa kalimat memerlukan karakter escape (`\`) agar dapat disimpan sebagai string dengan benar.
3. Hasil program harus sama persis dengan yang ada pada soal.

2.3 Analisis Program 3

a. Deskripsi Soal

Akar-akar suatu persamaan kuadrat $ax^2 + bx + c = 0$ dapat ditentukan menggunakan rumus berikut $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ dan dapat bertipe bilangan real maupun bilangan imajiner. Buatlah sebuah program untuk membaca koefisien-koefisien persamaan kuadrat lalu menyajikan akar-akar real persamaan tersebut (bila ada) atau menyebutkan bahwa persamaan tersebut hanya memiliki akar-akar imajiner. Atur tampilan agar akar-akar bilangan real ditampilkan dalam tiga desimal.

b. Tabel Input-Proses-Output

Bagian	Variabel / Langkah	Tipe Data	Keterangan
Input	a	float	koefisien x^2
	b	float	koefisien (x)
	c	float	konstanta
Proses	1	-	Membaca nilai a, b, dan c
	2	-	Menghitung diskriminan: ($D = b^2 - 4ac$)
	3	-	Melakukan pengecekan kondisi:
			• Jika ($D > 0$) → dua akar real berbeda
			• Jika ($D = 0$) → akar real kembar
			• Jika ($D < 0$) → akar imajiner

	4	-	Menghitung akar menggunakan rumus kuadrat
Output	Hasil akar	-	Nilai akar-akar persamaan kuadrat atau pesan bahwa akar bersifat imajiner

c. Batasan Masalah

1. Program hanya digunakan untuk menyelesaikan persamaan kuadrat berbentuk: $ax^2 + bx + c = 0$
2. Nilai koefisien aaa, bbb, dan ccc dimasukkan oleh pengguna dengan tipe data bilangan real (float).
3. Nilai aaa tidak boleh sama dengan 0 karena jika $a=0$, persamaan bukan merupakan persamaan kuadrat.
4. Program hanya menampilkan akar-akar real dari persamaan kuadrat.
5. Jika diskriminan $(D=b^2-4ac)$ bernilai negatif, program hanya menampilkan pesan bahwa persamaan memiliki akar imajiner tanpa menghitung nilai akar imajinernya.
6. Hasil akar-akar real ditampilkan dalam format tiga angka di belakang koma.
7. Program tidak menangani kesalahan input selain yang telah ditentukan pada batasan masalah.

d. Kondisi Khusus :

1. Jika nilai $a=0$, maka program menampilkan pesan bahwa persamaan bukan persamaan kuadrat.
2. Jika diskriminan $(D=b^2-4ac)$ lebih besar dari 0, maka persamaan memiliki dua akar real berbeda.
3. Jika diskriminan $(D=b^2-4ac)$ sama dengan 0, maka persamaan memiliki dua akar real kembar.
4. Jika diskriminan $(D=b^2-4ac)$ kurang dari 0, maka persamaan memiliki akar-akar imajiner.

5. Semua hasil akar real ditampilkan dengan tiga angka di belakang koma.

2.4 Analisis Program 4

a. Deskripsi Soal

Delapan digit pertama pada NIP seorang ASN berturut-turut menyatakan tahun lahir, bulan lahir, dan tanggal lahir ASN tersebut. Sebagai contoh, seorang ASN dengan NIP 199301212019031010 lahir pada tahun 1993, bulan 1 (Januari), tanggal 21. Buatlah sebuah program untuk menampilkan tanggal lahir ASN bila diberikan NIP ASN tersebut. Gunakan percabangan untuk memilih nama bulan yang sesuai.

b. Tabel Input-Proses-Output

KOMPONEN	KETERANGAN
INPUT	String NIP ASN (contoh: 199301212019031010).
PROSES	1. Memotong (slicing) string NIP digit 1–4 untuk Tahun Lahir. 2. Memotong string digit 5–6 untuk Bulan Lahir. 3. Memotong string digit 7–8 untuk Tanggal Lahir. 4. Melakukan percabangan (If-Else) untuk mengonversi angka bulan (01-12) menjadi nama bulan ("Januari" - "Desember").
OUTPUT	Teks yang menampilkan Tanggal Lahir lengkap (Tanggal, Nama Bulan, Tahun).

c. Batasan Masalah

- Input wajib berupa string berisi 18 digit angka tanpa spasi atau karakter khusus.

- Program hanya memproses 8 digit pertama NIP untuk mengambil tahun (4 digit), bulan (2 digit), dan tanggal (2 digit).
- Nilai bulan yang diproses dibatasi pada rentang 01 sampai 12.
- Program hanya menampilkan informasi tanggal lahir (tanggal, nama bulan, tahun). Sisa 10 digit NIP lainnya (informasi pengangkatan, jenis kelamin, dan nomor urut) diabaikan oleh program.

d. Kondisi Khusus :

- Penanganan jika panjang NIP kurang atau lebih dari 18 karakter.
- Penanganan jika pengguna memasukkan huruf, spasi, atau simbol.
- Kondisi jika digit bulan berada di luar rentang 01 sampai 12
- Kondisi jika digit tanggal berada di luar rentang 01 sampai 31 (atau tidak sesuai dengan jumlah hari pada bulan tersebut).

2.5 Analisis Program 5

a. Deskripsi Soal

Program ini digunakan untuk mencari cluster terdekat dari suatu titik pada ruang tiga dimensi berdasarkan jaraknya terhadap tiga pusat cluster yang telah ditentukan, yaitu A(2,1,3), B(1,-4,6), dan C(-2,3,-2). Program menerima koordinat titik U(x_1, x_2, x_3), kemudian menghitung jarak titik tersebut ke masing-masing pusat cluster menggunakan rumus Euclidean Distance.

$$\text{Jarak} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$$

Setelah seluruh jarak diperoleh, program membandingkan ketiga nilai jarak tersebut dan mengelompokkan titik U ke dalam cluster yang memiliki jarak paling kecil. Melalui program ini, konsep dasar klasifikasi berbasis jarak dapat diterapkan dengan memanfaatkan fungsi, operasi matematika, dan percabangan dalam bahasa pemrograman Python maupun R.

b. Tabel Input-Proses-Output

Komponen	Keterangan
Input	Koordinat titik U berupa 3 bilangan riil yang dimasukkan sebagai satu kesatuan (<i>tuple/list</i>).

Proses	1. Menyimpan koordinat pusat cluster tetap: A (2,1,3), B (1,-4,6) dan C (-2,3,-2) sebagai konstanta. 2. Menghitung jarak Euclidean dari titik U ke pusat A, B, dan C menggunakan rumus: $\text{Jarak} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$ 3. Membandingkan ketiga hasil jarak untuk mencari nilai minimum.
Output	String teks yang menyatakan nama cluster tempat titik U bernaung (Cluster A, Cluster B, atau Cluster C) serta nilai jaraknya.

c. Batasan Masalah

- Program menggunakan tiga pusat cluster yang nilainya tetap:
 - A = (2, 1, 3)
 - B = (1, -4, 6)
 - C = (-2, 3, -2)
- Nilai x1, x2, dan x3 berupa bilangan numerik (integer atau desimal).
- Perhitungan jarak menggunakan rumus Euclidean 3 dimensi:
$$\text{Jarak} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$$
- Program hanya menentukan cluster berdasarkan jarak terkecil.

d. Kondisi Khusus

- Titik U memiliki satu jarak terkecil yang jelas terhadap salah satu cluster.
- Program mengembalikan Cluster A, B, atau C.
- Kondisi Khusus
- Titik U berada sangat dekat atau tepat pada pusat suatu cluster.

2.6 Analisis Program 6

a. Deskripsi Soal

b. Tabel Input-Proses-Output

No	Input	Proses	Output
1	Proporsi sampel ($\hat{p}$)	Program membaca nilai proporsi sampel yang dimasukkan pengguna	Nilai proporsi sampel tersimpan
2	Nilai alpha (α)	Program membaca tingkat signifikansi yang dipilih pengguna	Nilai alpha tersimpan
3	Ukuran sampel (n)	Program membaca jumlah sampel	Nilai ukuran sampel tersimpan
4	$\hat{p}$	Program memeriksa apakah nilai proporsi berada pada interval 0 sampai 1	Pesan error jika $\hat{p} < 0$ atau $\hat{p} > 1$
5	Nilai alpha	Program menentukan nilai z berdasarkan alpha	$z = 1,645$ untuk $\alpha = 0,10$ atau $z = 1,96$ untuk $\alpha = 0,05$
6	$\hat{p}$, n, dan z	Program menghitung margin of error menggunakan rumus: $z \sqrt{(\hat{p}(1-\hat{p})/n)}$	Nilai margin of error
7	Margin of error dan $\hat{p}$	Program menghitung batas bawah dan batas atas interval konfidensi	Interval konfidensi
8	Semua hasil perhitungan	Program menampilkan hasil interval konfidensi	Tampilan hasil: batas bawah < p < batas atas

c. Batasan Masalah

1. Program hanya digunakan untuk menghitung interval konfidensi proporsi populasi.
2. Nilai proporsi sampel ($\hat{p}$) harus berada antara 0 sampai 1.
3. Nilai alpha yang digunakan hanya 0,10 dan 0,05.
4. Ukuran sampel (n) harus berupa bilangan positif.
5. Program dibuat menggunakan bahasa pemrograman R.

d. Kondisi Khusus

1. Jika nilai proporsi sampel ($\hat{p} < 0$) atau ($\hat{p} > 1$), maka program menampilkan pesan error.
2. Jika nilai alpha selain 0,10 atau 0,05, maka program menampilkan pesan bahwa alpha tidak valid.
3. Jika ukuran sampel ($n \leq 0$), maka program menampilkan pesan error karena ukuran sampel harus positif.

2.7 Pembagian Tugas Kelompok

Tugas	Nama
Program 1	Nadien Dwi Anggraeni
Program 2	Khalisa Irfani
Program 3	Safira Aulia
Program 4	Meyluna Prachi Sri Rumondang Purba
Program 5	Khalisa Irfani, Meyluna Prachi Sri Rumondang Purba
Program 6	Safira Aulia
Laporan	Bersama
Presentasi	Bersama
Github	Bersama

BAB III

PSEUDOCODE DAN FLOWCHART

3.1 Pseudocode dan Flowchart program 1

a. Pseudocode

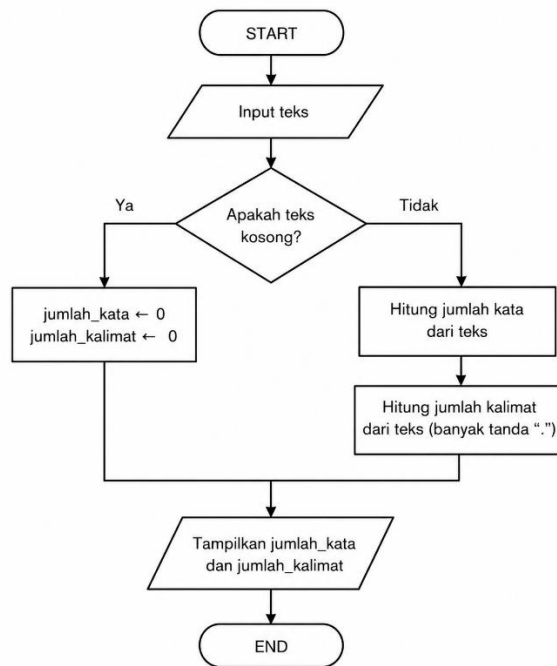
```
ALGORITMA MENGHITUNG JUMLAH KATA DAN JUMLAH KALIMAT
DEKLARASI
teks:string
teks_trim:string
jumlah_kata:integer
jumlah_kalimat : integer
ALGORITMA
START
INPUT teks

IF teks kosong THEN
    jumlah_kata ← 0
    jumlah_kalimat ← 0
ELSE
    jumlah_kata ← banyak kata dalam teks
    jumlah_kalimat ← banyak tanda titik dalam teks
ENDIF

TAMPILKAN jumlah kata
TAMPILKAN jumlah kalimat

END
```

b. Flowchart



3.2 Pseudocode dan Flowchart program 2

a. Pseudocode

```

Algoritma Program 2

DEKLARASI
  K1: str
  K2: str
  K3: str
IMPLEMENTASI
  START
  Proses K1, K2, K3, K4
  K1="Saya tak `kan menyerah."
  K2='Ia berkata,"Aku menyayangimu."'
  K3="\Coba jelaskan pengertian `cross-validation` dalam Machine Learning!\\""
  K4="Surat keputusan itu bernomor 62/UN.34/19/2023
  ."
  print(K1)
  print(K2)
  
```



```

print (K3)

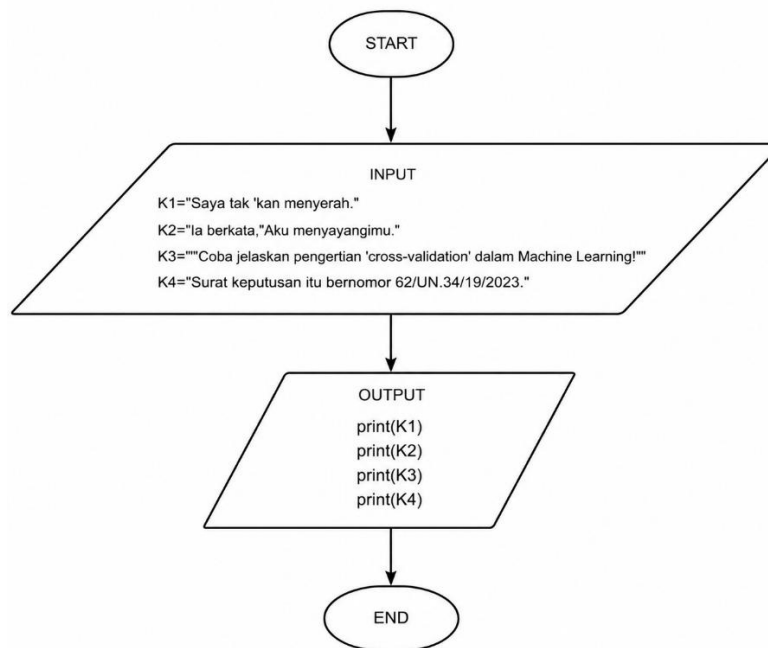
print (K4)

END

```

b. Flowchart

Flowchart program 2



3.3 Pseudocode dan Flowchart Program 3

a. Pseudocode

```

Start
Input a
Input b
Input c
Hitung D = b^2 - 4*a*c
Jika D > 0 maka
Hitung x1 = (-b + √D) / (2*a)
Hitung x2 = (-b - √D) / (2*a)
Tampilkan x1 dan x2
Jika D = 0 maka

```

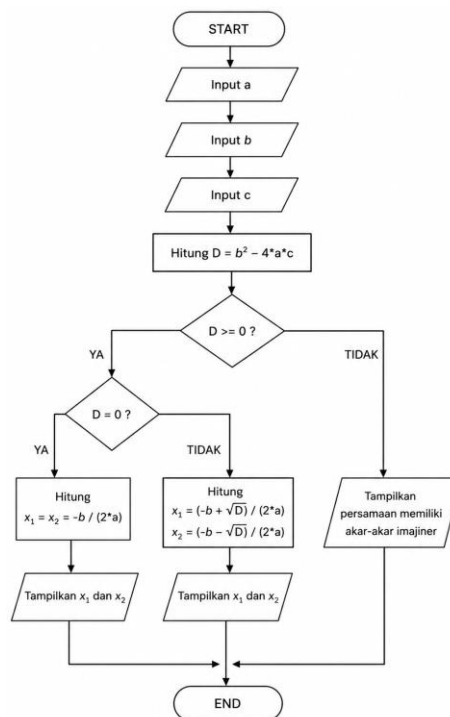
```

Hitung  $x_1 = x_2 = -b / (2 \cdot a)$ 
Tampilkan  $x_1$  dan  $x_2$ 
Jika  $D < 0$  maka
Tampilkan "Persamaan memiliki akar-akar imajiner"
End

```

b. Flowchart

Flowchart program 3



3.4 Pseudocode dan Flowchart program 4

a. Pseudocode

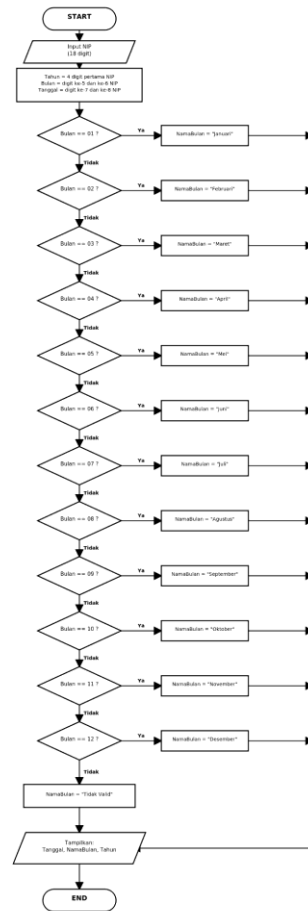
```

START
Input NIP
Jika panjang NIP < 8
Tampilkan "NIP tidak valid"
Selesai
Ambil 4 digit pertama sebagai tahun
Ambil digit ke-5 dan ke-6 sebagai bulan
Ambil digit ke-7 dan ke-8 sebagai tanggal

```

```
Gunakan percabangan:
Jika bulan = "01"
nama_bulan = "Januari"
Jika bulan = "02"
nama_bulan = "Februari"
Jika bulan = "03"
nama_bulan = "Maret"
Jika bulan = "04"
nama_bulan = "April"
Jika bulan = "05"
nama_bulan = "Mei"
Jika bulan = "06"
nama_bulan = "Juni"
Jika bulan = "07"
nama_bulan = "Juli"
Jika bulan = "08"
nama_bulan = "Agustus"
Jika bulan = "09"
nama_bulan = "September"
Jika bulan = "10"
nama_bulan = "Oktober"
Jika bulan = "11"
nama_bulan = "November"
Jika bulan = "12"
nama_bulan = "Desember"
Selain itu
nama_bulan = "Bulan tidak valid"
Tampilkan tanggal, nama_bulan, tahun
END
```

b. Flowchart



3.5 Pseudocode dan Flowchart program 5

a. Pseudocode

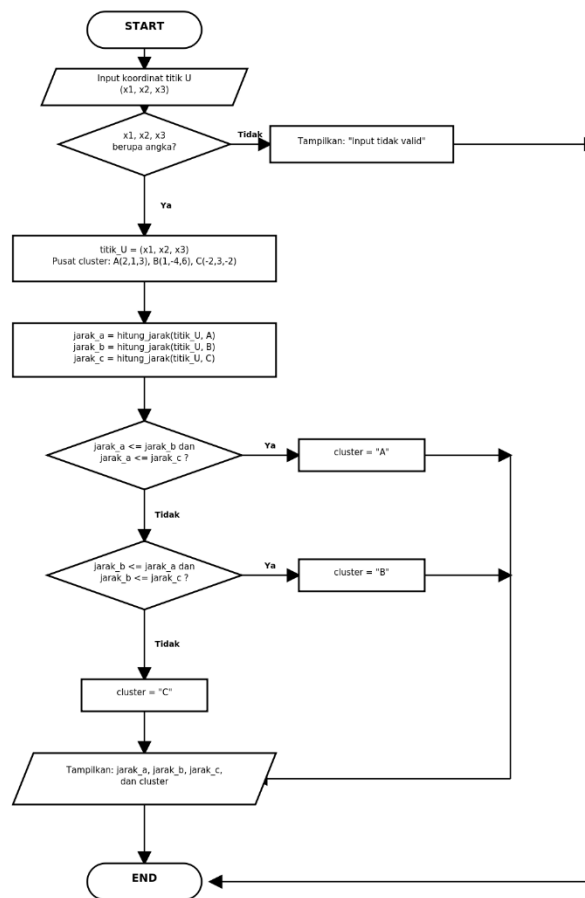
```

START
Input x1, x2, x3
Jika x1 bukan angka atau x2 bukan angka atau x3 bukan
angka
Tampilkan "Input harus berupa angka/bilangan real"
Selesai

Simpan koordinat pusat_A = (2, 1, 3)
Simpan koordinat pusat_B = (1, -4, 6)
Simpan koordinat pusat_C = (-2, 3, -2)
Hitung jarak ke pusat_A:
jarak_A = akar((x1 - 2)^2 + (x2 - 1)^2 + (x3 - 3)^2)
  
```

```
Hitung jarak ke pusat_B:
jarak_b = akar((x1 - 1)^2 + (x2 - (-4))^2 + (x3 - 6)^2)
Hitung jarak ke pusat_C:
jarak_C = akar((x1 - (-2))^2 + (x2 - 3)^2 + (x3 - (-2))^2)
Gunakan percabangan:
Jika jarak_A <= jarak_B dan jarak_A <= jarak_C
nama_cluster = "Cluster A"
Jika jarak_B <= jarak_A dan jarak_B <= jarak_C
nama_cluster = "Cluster B"
Jika jarak_C <= jarak_A dan jarak_C <= jarak_B
nama_cluster = "Cluster C"
Tampilkan nama_cluster, jarak_A, jarak_B, jarak_C
END
```

b.Flowchart



3.6 Pseudocode dan Flowchart program 6

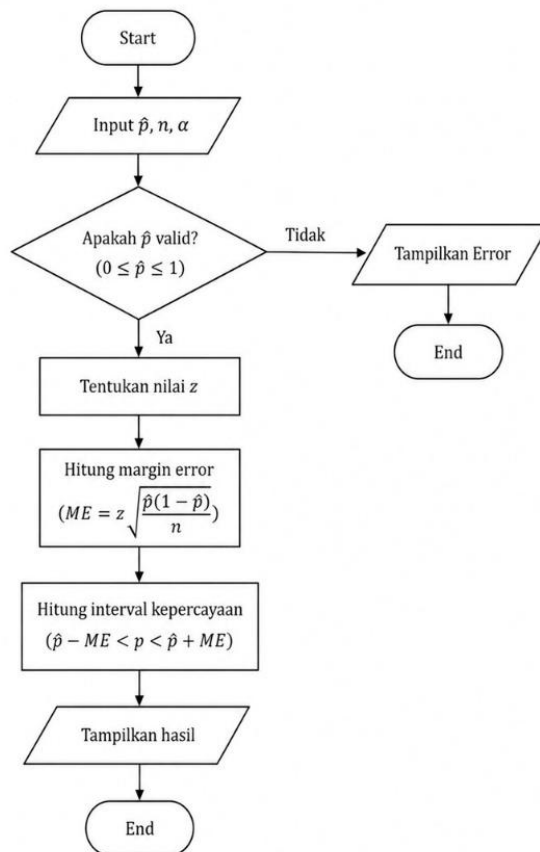
a. Pseudocode

```

Start
Input nilai  $\hat{p}$ 
Input ukuran sampel n
Input nilai  $\alpha$ 
Jika  $\hat{p} < 0$  atau  $\hat{p} > 1$ :
  Tampilkan pesan kesalahan
Jika tidak:
  Jika  $\alpha = 0.10$  maka  $z = 1.645$ 
  Jika  $\alpha = 0.05$  maka  $z = 1.96$ 
  Hitung margin error
  Hitung batas bawah
  
```

Hitung batas atas
Tampilkan interval kepercayaan
End

b. Flowchart



BAB IV

IMPLEMENTASI PROGRAM

4.1 Source code Python dan R Program 1

a. Source code Python

```
# Program Menghitung Jumlah Kata dan Jumlah Kalimat

teks = input("Masukkan teks: ")

if teks.strip() == "":
    jumlah_kata = 0
    jumlah_kalimat = 0
else:
    jumlah_kata = len(teks.split())
    jumlah_kalimat = len(teks.split("."))-1

print("\nTeks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")
```

b. Source code R

```
{
  # Membaca input teks dari pengguna
  cat("Masukkan teks: ")
  teks <- readline()

  # Validasi jika input kosong (setelah menghapus spasi
  di ujung)
  teks_trim <- trimws(teks)

  if (teks_trim == "") {
    jumlah_kata <- 0
```



```

    jumlah_kalimat <- 0
  } else {
    # Menghitung jumlah kata dengan memisahkan
    berdasarkan spasi menggunakan strsplit
    kata_list <- strsplit(teks_trim, "\\s+")[[1]]
    jumlah_kata <- length(kata_list)

    # Menghitung jumlah kalimat dengan menghitung
    frekuensi tanda titik (.)
    pencocokan_titik <- gregexpr("\\.", teks)[[1]]

    if (pencocokan_titik[1] == -1) {
      jumlah_kalimat <- 0
    } else {
      jumlah_kalimat <- length(pencocokan_titik)
    }
  }

  # Menampilkan hasil output
  cat(paste("Teks tersebut memuat", jumlah_kalimat,
    "kalimat dan", jumlah_kata, "kata.\n"))
}

```

4.2 Source code Python dan R Program 2

a. Source code Python

```

# -*- coding: utf-8 -*-
"""untitled6.ipynb

Automatically generated by Colab.

Original file is located at

```

```

https://colab.research.google.com/gist/khalisairfani-netizen/b2c4671774fcb5d54edd0fcd75d2ba7a/untitled6.ipynb
b
"""

K1 = "Saya tak 'kan menyerah."
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)

```

b. Source code R

```

K1 <- "Saya tak 'kan menyerah."
K2 <- 'Ia berkata, "Aku menyayangimu."'
K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."

cat(K1, "\n")
## Saya tak 'kan menyerah.
cat(K2, "\n")
## Ia berkata, "Aku menyayangimu."
cat(K3, "\n")
## "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"

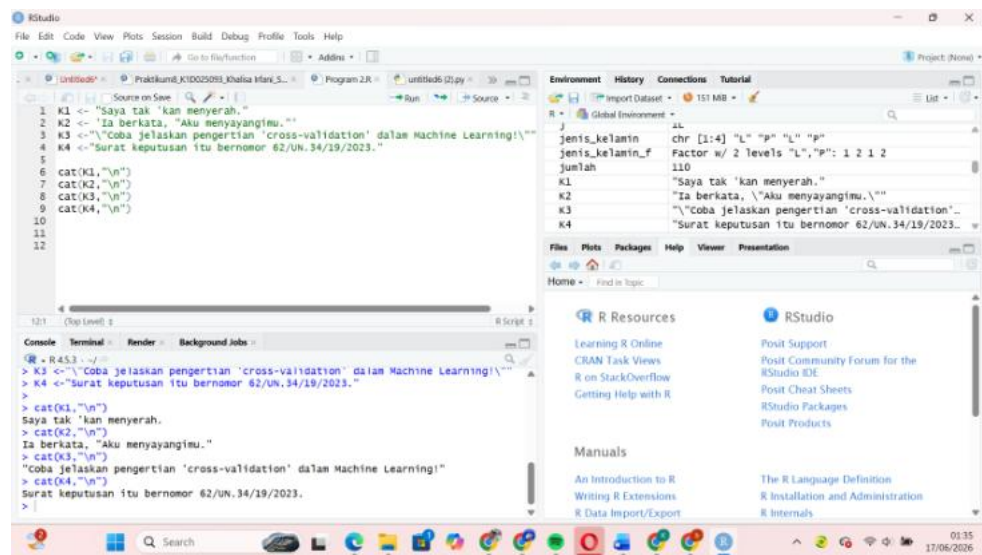
```

```
cat(K4, "\n")
```

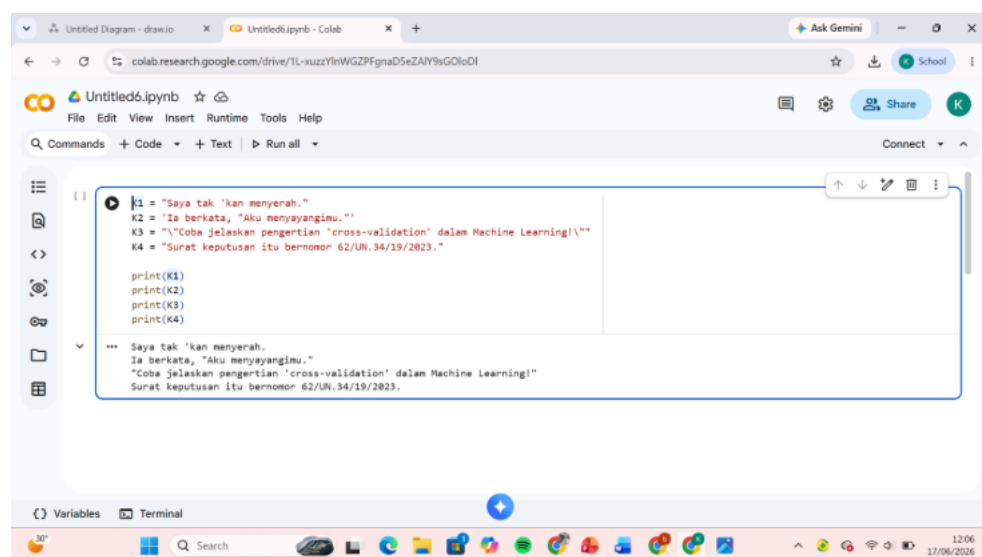
```
## Surat keputusan itu bernomor 62/UN.34/19/2023.
```

c. Screenshoot

Screenshoot program 2 R:



Screenshoot program 2 python:



4.3 Source code Python dan R Program 3

a. Source Code Python

```
import math
def akar_kuadrat(a, b, c):
    D = b**2 - 4*a*c
    if D > 0:
        x1 = (-b + math.sqrt(D)) / (2*a)
        x2 = (-b - math.sqrt(D)) / (2*a)

        print(f"Akar pertama = {x1:.3f}")
        print(f"Akar kedua    = {x2:.3f}")

    elif D == 0:
        x = -b / (2*a)

        print(f"Akar kembar = {x:.3f}")

    else:
        print("Persamaan memiliki akar-akar imajiner")

a = float(input("Masukkan a : "))
b = float(input("Masukkan b : "))
c = float(input("Masukkan c : "))

akar_kuadrat(a,b,c)
```

b. Source Code R

```
akar_kuadrat <- function(a,b,c){
  D <- b^2 - 4*a*c
  if(D > 0){
```

```

x1 <- (-b + sqrt(D))/(2*a)
x2 <- (-b - sqrt(D))/(2*a)
cat(sprintf("Akar pertama = %.3f\n",x1))
cat(sprintf("Akar kedua    = %.3f\n",x2))

} else if(D == 0){
  x <- -b/(2*a)

  cat(sprintf("Akar kembar = %.3f\n",x))
} else {
  cat("Persamaan memiliki akar-akar imajiner\n")
}
}
a <- as.numeric(readline("Masukkan a : "))
b <- as.numeric(readline("Masukkan b : "))
c <- as.numeric(readline("Masukkan c : "))

akar_kuadrat(a,b,c)

```

4.4 Source code Python dan R Program 4

a. Source code Python

```

def tampilkan_tanggal_lahir(nip):
    if len(nip) < 8:
        print("NIP tidak valid")
    else:
        tahun = nip[0:4]
        bulan = nip[4:6]
        tanggal = nip[6:8]
        if bulan == "01":
            nama_bulan = "Januari"
        elif bulan == "02":
            nama_bulan = "Februari"

```

```

elif bulan == "03":
    nama_bulan = "Maret"
elif bulan == "04":
    nama_bulan = "April"
elif bulan == "05":
    nama_bulan = "Mei"
elif bulan == "06":
    nama_bulan = "Juni"
elif bulan == "07":
    nama_bulan = "Juli"
elif bulan == "08":
    nama_bulan = "Agustus"
elif bulan == "09":
    nama_bulan = "September"
elif bulan == "10":
    nama_bulan = "Oktober"
elif bulan == "11":
    nama_bulan = "November"
elif bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan tidak valid"
print("Tanggal Lahir ASN:", tanggal, nama_bulan, tahun)
# Memanggil fungsi
tampilkan_tanggal_lahir("199301212019031010")

```

b. Source Code R

```

tampilkan_tanggal_lahir <- function(nip) {
  if (nchar(nip) < 8) {
    cat("NIP tidak valid\n")
  } else {

```

```

tahun<- substr(nip, 1, 4)
bulan<- substr(nip, 5, 6)
tanggal <- substr(nip, 7, 8)
if (bulan == "01") {
nama_bulan <- "Januari"
} else if (bulan == "02") {
nama_bulan <- "Februari"
} else if (bulan == "03") {
nama_bulan <- "Maret"
} else if (bulan == "04") {
nama_bulan <- "April"
} else if (bulan == "05") {
nama_bulan <- "Mei"
} else if (bulan == "06") {
nama_bulan <- "Juni"
} else if (bulan == "07") {
nama_bulan <- "Juli"
} else if (bulan == "08") {
nama_bulan <- "Agustus"
} else if (bulan == "09") {
nama_bulan <- "September"
} else if (bulan == "10") {
nama_bulan <- "Oktober"
} else if (bulan == "11") {
nama_bulan <- "November"
} else if (bulan == "12") {
nama_bulan <- "Desember"
} else {
nama_bulan <- "Bulan tidak valid"
}

```

<pre> cat("Tanggal ASN:", tanggal, nama_bulan, tahun, "\n") } } tampilkan_tanggal_lahir("199301212019031010") </pre>	Lahir
--	-------

4.5 Source code Python dan R Program 5

a. Source code Python

```

# -*- coding: utf-8 -*-
"""Untitled0.ipynb
Automatically generated by Colab.
Original file is located at
https://colab.research.google.com/drive/1ukURRKTwil7ituU
nTtR7vDiIPAzicQwk
"""
import math
# Pusat cluster
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)
# Titik U
U = (1, 2, 3)
# Menghitung jarak ke masing-masing cluster
jarak_A = math.sqrt((U[0] - A[0])**2 + (U[1] - A[1])**2
+ (U[2] - A[2])**2)
jarak_B = math.sqrt((U[0] - B[0])**2 + (U[1] - B[1])**2
+ (U[2] - B[2])**2)
jarak_C = math.sqrt((U[0] - C[0])**2 + (U[1] - C[1])**2
+ (U[2] - C[2])**2)
print("Jarak ke A =", jarak_A)
print("Jarak ke B =", jarak_B)
print("Jarak ke C =", jarak_C)

```



```
# Menentukan cluster terdekat
if jarak_A < jarak_B and jarak_A < jarak_C:
print("Titik U termasuk Cluster A")
elif jarak_B < jarak_A and jarak_B < jarak_C:
print("Titik U termasuk Cluster B")
else:
print("Titik U termasuk Cluster C")
```

b. Source code Python

```
# Titik pusat cluster
A <- c(2, 1, 3)
B <- c(1, -4, 6)
C <- c(-2, 3, -2)
# Input data
x1 <- 1
x2 <- 2
x3 <- 3
U <- c(x1, x2, x3)
# Jarak ke cluster A
jarakA <- sqrt((U[1]-A[1])^2 +
(U[2]-A[2])^2 +
(U[3]-A[3])^2)
# Jarak ke cluster B
jarakB <- sqrt((U[1]-B[1])^2 +
(U[2]-B[2])^2 +
(U[3]-B[3])^2)
# Jarak ke cluster C
jarakC <- sqrt((U[1]-C[1])^2 +
(U[2]-C[2])^2 +
(U[3]-C[3])^2)
```

```

# Menentukan cluster
if (jarakA <= jarakB && jarakA <= jarakC) {
  cluster <- "A"
} else if (jarakB <= jarakA && jarakB <= jarakC) {
  cluster <- "B"
} else {
  cluster <- "C"
}
cat("Jarak ke A =", jarakA, "\n")
## Jarak ke A = 1.414214
cat("Jarak ke B =", jarakB, "\n")
## Jarak ke B = 6.708204
cat("Jarak ke C =", jarakC, "\n")
## Jarak ke C = 5.91608
cat("Cluster =", cluster)
## Cluster = A

```

4.6 Source code Python dan R Program 6

a. Source code Python

```

# Input data
p <- 0.6
n <- 100
alpha <- 0.05
# Menentukan nilai z
if (alpha == 0.05) {
  z <- 1.96
} else if (alpha == 0.10) {
  z <- 1.645
} else {
  print("Alpha tidak valid")
}
# Menghitung margin of error

```

```
margin_error <- z * sqrt((p * (1 - p)) / n)
```

```
# Menghitung interval kepercayaan
```

```
batas_bawah <- p - margin_error
```

```
batas_atas <- p + margin_error
```

```
# Menampilkan hasil
```

```
cat("Interval Kepercayaan Proporsi Populasi\n")
```

```
cat("-----\n")
```

```
cat("Batas bawah =", round(batas_bawah, 4), "\n")
```

```
cat("Batas atas =", round(batas_atas, 4), "\n")
```

b. Source code R

```
import math
# Input data
p = float(input("Masukkan proporsi sampel ( $p^{\wedge}$ ): "))
n = int(input("Masukkan ukuran sampel (n): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi nilai proporsi
if p < 0 or p > 1:
    print("Error: Nilai proporsi harus antara 0 dan 1")
else:

    # Menentukan nilai z berdasarkan alpha
    if alpha == 0.10:
        z = 1.645
```

```

elif alpha == 0.05:
    z = 1.96
else:
    print("Error: Alpha harus 0.10 atau 0.05")
    exit()

# Menghitung margin of error
margin_error = z * math.sqrt((p * (1 - p)) / n)

# Menghitung batas bawah dan batas atas
batas_bawah = p - margin_error
batas_atas = p + margin_error

# Menampilkan hasil
print("\nInterval Kepercayaan Proporsi Populasi")
print("-----")
print("Batas bawah =", round(batas_bawah, 4))
print("Batas atas  =", round(batas_atas, 4))

```

BAB V

PENJELASAN LINE-BY-LINE CODE

5.1 Tabel penjelasan setiap baris kode Python dan R Program 1

a. Baris kode program Python

No.	Baris kode	Keterangan
1.	teks = input("Masukkan teks:")	Membaca teks yang dimasukkan pengguna dan menyimpannya pada variabel teks.
2.	if teks.strip() == "":	Memeriksa apakah input kosong setelah spasi di awal dan akhir dihapus.
3.	jumlah_kata = 0	Jika teks kosong maka jumlah kata bernilai 0.
4.	jumlah_kalimat = 0	Jika teks kosong maka jumlah kalimat bernilai 0.
5.	else:	Menjalankan proses jika teks tidak kosong.
6.	jumlah_kata = len(teks.split())	Memecah teks menjadi beberapa kata berdasarkan spasi kemudian menghitung banyaknya kata.
7.	jumlah_kalimat = len(teks.split(".")) - 1	Menghitung banyak tanda titik sebagai penanda akhir kalimat.
8.	print("\nTeks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")	Menampilkan hasil perhitungan jumlah kalimat dan jumlah kata kepada pengguna dalam bentuk kalimat yang mudah dibaca. Karakter \n digunakan untuk membuat baris baru sebelum hasil ditampilkan agar output terlihat lebih rapi. Nilai dari variabel jumlah_kalimat dan jumlah_kata akan disisipkan ke dalam kalimat output.

b. Baris kode program R

No.	Baris kode	Keterangan
1.	{	Menandai awal blok program.
2.	# Membaca input teks dari pengguna	Komentar yang menjelaskan bahwa program akan menerima masukan teks dari pengguna.
3.	cat("Masukkan teks: ")	Menampilkan pesan "Masukkan teks:" sebagai petunjuk bagi pengguna untuk memasukkan teks.
4.	teks <- readline()	Membaca teks yang dimasukkan pengguna melalui keyboard dan menyimpannya ke dalam variabel teks.
5.	# Validasi jika input kosong (setelah menghapus spasi di ujung)	Komentar yang menjelaskan bahwa program akan memeriksa apakah input kosong atau tidak.
6.	teks_trim <- trimws(teks)	Menghapus spasi di awal dan akhir teks untuk memastikan validasi input dilakukan dengan benar.
7.	if (teks_trim == "") {	Memeriksa apakah teks yang dimasukkan kosong setelah spasi dihapus.
8.	jumlah_kata <- 0	Memberikan nilai 0 pada variabel jumlah_kata jika input kosong.
9.	jumlah_kalimat <- 0	Memberikan nilai 0 pada variabel jumlah_kalimat jika input kosong.
10.	} else {	Menjalankan proses perhitungan jika input tidak kosong.
11.	# Menghitung jumlah kata dengan memisahkan	Komentar yang menjelaskan proses perhitungan jumlah kata.

	berdasarkan spasi menggunakan strsplit	
12.	kata_list <- strsplit(teks_trim, "\\s+")[1]	Memisahkan teks menjadi beberapa kata berdasarkan spasi dan menyimpannya dalam variabel kata_list.
13.	jumlah_kata <- length(kata_list)	Menghitung banyaknya kata dalam kata_list.
14.	# Menghitung jumlah kalimat dengan menghitung frekuensi tanda titik (.)	Komentar yang menjelaskan proses perhitungan jumlah kalimat.
15.	pencocokan_titik <- gregexpr("\\.", teks)[1]	Mencari seluruh posisi tanda titik (.) dalam teks dan menyimpannya ke variabel pencocokan_titik.
16.	if (pencocokan_titik[1] == -1) {	Memeriksa apakah tidak ditemukan tanda titik dalam teks. Nilai -1 menunjukkan tidak ada hasil pencarian.
17.	jumlah_kalimat <- 0	Memberikan nilai 0 pada jumlah kalimat jika tidak ditemukan tanda titik.
18.	} else {	Menjalankan perhitungan jumlah kalimat jika ditemukan tanda titik.
19.	jumlah_kalimat <- length(pencocokan_titik)	Menghitung jumlah tanda titik yang ditemukan. Sesuai asumsi soal, setiap titik dianggap sebagai akhir satu kalimat.
20.	}	Mengakhiri percabangan pemeriksaan tanda titik.

21.	}	Mengakhiri percabangan pemeriksaan input kosong.
22.	# Menampilkan hasil output	Komentar yang menjelaskan bahwa program akan menampilkan hasil perhitungan.
23.	cat(paste("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.\n"))	Menampilkan hasil jumlah kalimat dan jumlah kata dalam bentuk kalimat yang mudah dipahami pengguna.
24.	}	Menandai akhir blok program.

5.2 Tabel penjelasan setiap baris kode Python dan R Program 2

a. Baris kode program Python

No	Baris kode program python	Keterangan
1.	K1 = "Saya tak 'kan menyerah."	Membuat variabel K1 dan menyimpan kalimat "Saya tak 'kan menyerah." sebagai data bertipe string.
2.	K2 = 'Ia berkata, "Aku menyayangimu."'	Membuat variabel K2 dan menyimpan kalimat yang mengandung tanda petik ganda sebagai bagian dari isi string.
3.	K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""	Membuat variabel K3 dan menyimpan kalimat yang diawali dan diakhiri tanda petik ganda. Karakter escape (\\) digunakan agar tanda petik ganda dapat ditampilkan dalam output.

4.	K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	Membuat variabel K4 dan menyimpan teks nomor surat keputusan sebagai string.
5.	print(K1)	Menampilkan isi variabel K1 ke layar
6.	print(K2)	Menampilkan isi variabel K2 ke layar
7.	print(K3)	Menampilkan isi variabel K3 ke layar dengan tanda petik ganda yang menjadi bagian dari teks.
8.	Print(K4)	Menampilkan isi variabel K4 ke layar.

b. Baris kode program R

No	Baris kode program R	Keterangan
1.	K1 <- "Saya tak 'kan menyerah."	Membuat objek K1 dan menyimpan kalimat "Saya tak 'kan menyerah." sebagai data karakter (character string)
2.	K2 <- 'Ia berkata, "Aku menyayangimu."'	Membuat objek K2 dan menyimpan kalimat yang mengandung tanda petik ganda sebagai bagian dari teks.
3.	K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""	Membuat objek K3 dan menyimpan kalimat yang diawali dan diakhiri tanda petik ganda. Karakter escape (\\) digunakan agar tanda petik ganda dapat dicetak sebagai isi teks.
4.	K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."	Membuat objek K4 dan menyimpan teks nomor surat keputusan sebagai data karakter.

5.	<code>cat(K1, "\n")</code>	Menampilkan isi objek K1 ke konsol dan menambahkan karakter pindah baris (\n) setelah output ditampilkan.
6.	<code>cat(K2, "\n")</code>	Menampilkan isi objek K2 ke konsol dan berpindah ke baris berikutnya.
7.	<code>cat(K3, "\n")</code>	Menampilkan isi objek K3 ke konsol, termasuk tanda petik ganda yang merupakan bagian dari teks.
8.	<code>cat(K4, "\n")</code>	Menampilkan isi objek K4 ke konsol dan mengakhiri output pada baris baru.

5.3 Tabel penjelasan setiap baris kode Python dan R Program 3

a. Baris kode program Python

Baris Kode Program Python	Penjelasan Logis
<code>import math</code>	Memanggil library matematika untuk menggunakan fungsi akar kuadrat.
<code>def akar_kuadrat(a,b,c):</code>	Membuat fungsi agar proses perhitungan dapat digunakan kembali.
<code>D = b**2 - 4ac</code>	Menghitung diskriminan untuk menentukan jenis akar.
<code>if D > 0:</code>	Memeriksa apakah terdapat dua akar real berbeda.
<code>x1 = ...</code>	Menghitung akar pertama dengan rumus kuadrat.
<code>x2 = ...</code>	Menghitung akar kedua dengan rumus kuadrat.
<code>print(...)</code>	Menampilkan hasil dengan tiga angka desimal.
<code>elif D == 0:</code>	Memeriksa apakah akar kembar.

$x = -b/(2*a)$	Menghitung akar tunggal.
else	Kondisi ketika diskriminan negatif.
print("...")	Memberikan informasi bahwa akar bersifat imajiner.
input()	Membaca nilai dari pengguna.
akar_kuadrat(a,b,c)	Menjalankan fungsi yang telah dibuat.

b. Baris kode program R

Baris Kode	Penjelasan Logis
akar_kuadrat <- function(...)	Membuat fungsi perhitungan akar persamaan kuadrat.
$D <- b^2 - 4ac$	Menghitung diskriminan.
if($D > 0$)	Memeriksa apakah akar real berbeda.
sqrt(D)	Mengambil akar kuadrat dari diskriminan.
$x1 <- \dots$	Menghitung akar pertama.
$x2 <- \dots$	Menghitung akar kedua.
sprintf()	Mengatur tampilan hingga tiga desimal.
else if($D == 0$)	Memeriksa akar kembar.
$x <- -b/(2*a)$	Menghitung akar kembar.
else	Kondisi diskriminan negatif.
readline()	Membaca input pengguna.
as.numeric()	Mengubah input menjadi tipe numerik.

5.4 Tabel penjelasan setiap baris kode Python dan R Program 4

a. Baris Kode Program Python

No	Baris Kode Python	Keterangan
1.	<code>def tampilkan_tanggal_lahir(nip):</code>	Mendefinisikan sebuah fungsi bernama <code>tampilkan_tanggal_lahir</code> yang menerima satu parameter input bernama <code>nip</code> . Fungsi ini dibuat agar kode program menjadi lebih modular dan rapi.
2.	<code>if len(nip) < 8:</code>	[Validasi Input] Melakukan pengecekan kondisi apakah panjang string dari variabel <code>nip</code> kurang dari 8 karakter, karena 8 karakter pertama adalah batas minimal untuk membaca data kelahiran.
3.	<code>print("NIP tidak valid")</code>	Menampilkan pesan kesalahan kepada pengguna di layar jika data NIP yang dimasukkan cacat atau kurang dari 8 digit sebagai bentuk penanganan kondisi khusus (<i>error handling</i>).
4.	<code>else:</code>	Menangani alur logika alternatif jika kondisi pada baris ke-2 tidak terpenuhi (artinya panjang string <code>nip</code> sudah memenuhi syarat minimal yaitu 8 karakter atau lebih).
5.	<code>tahun = nip[0:4]</code>	[Proses Slicing] Mengekstrak karakter string dari indeks ke-0 hingga sebelum indeks ke-4 (4 digit pertama) pada variabel <code>nip</code> untuk mengambil data tahun lahir ASN, lalu menyimpannya ke variabel <code>tahun</code> .
6.	<code>bulan = nip[4:6]</code>	[Proses Slicing] Mengekstrak karakter string dari indeks ke-4 hingga sebelum indeks ke-6 (digit ke-5 dan ke-6) pada variabel <code>nip</code> untuk mendapatkan kode angka bulan lahir ASN, lalu menyimpannya ke variabel <code>bulan</code> .
7.	<code>tanggal <- substr(nip, 7, 8)</code>	[Proses Slicing] Mengekstrak potongan teks dari karakter posisi ke-7 sampai posisi ke-8 pada variabel <code>nip</code> menggunakan fungsi <code>substr()</code> untuk mendapatkan 2 digit komponen angka tanggal lahir ASN, kemudian menyimpannya ke variabel <code>tanggal</code> .

8.	<code>if (bulan == "01") {</code>	[Percabangan Kondisional] Memeriksa apakah isi dari variabel bulan bernilai sama dengan string "01". Jika benar, program akan mengeksekusi blok kode di dalam kurung kurawal <code>}</code> untuk mendefinisikan bulan Januari.
9.	<code>nama_bulan = "Januari"</code>	Mengisi variabel <code>nama_bulan</code> dengan objek string bertuliskan "Januari" karena kondisi pada baris ke-8 terpenuhi.
10.	<code>elif bulan == "02":</code>	(<i>Else If</i>) Memeriksa kondisi alternatif jika komponen angka bulan bernilai "02". Jika benar, variabel <code>nama_bulan</code> akan diisi dengan teks "Februari".
11.	<code>nama_bulan = "Februari"</code>	Mengisi variabel <code>nama_bulan</code> dengan objek string bertuliskan "Februari".
12.	<code>elif bulan == "03":</code>	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "03". Jika benar, variabel <code>nama_bulan</code> akan diisi dengan teks "Maret".
13.	<code>nama_bulan = "Maret"</code>	Mengisi variabel <code>nama_bulan</code> dengan objek string bertuliskan "Maret".
14.	<code>elif bulan == "04":</code>	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "04". Jika benar, variabel <code>nama_bulan</code> akan diisi dengan teks "April".
15.	<code>nama_bulan = "April"</code>	Mengisi variabel <code>nama_bulan</code> dengan objek string bertuliskan "April".
16.	<code>elif bulan == "05":</code>	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "05". Jika benar, variabel <code>nama_bulan</code> akan diisi dengan teks "Mei".
17.	<code>nama_bulan = "Mei"</code>	Mengisi variabel <code>nama_bulan</code> dengan objek string bertuliskan "Mei".

18.	elif bulan == "06":	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "06". Jika benar, variabel nama_bulan akan diisi dengan teks "Juni".
19.	nama_bulan = "Juni"	Mengisi variabel nama_bulan dengan objek string bertuliskan "Juni".
20.	elif bulan == "07":	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "07". Jika benar, variabel nama_bulan akan diisi dengan teks "Juli".
21.	nama_bulan = "Juli"	Mengisi variabel nama_bulan dengan objek string bertuliskan "Juli".
22.	elif bulan == "08":	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "08". Jika benar, variabel nama_bulan akan diisi dengan teks "Agustus".
23.	nama_bulan = "Agustus"	Mengisi variabel nama_bulan dengan objek string bertuliskan "Agustus".
24.	elif bulan == "09":	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "09". Jika benar, variabel nama_bulan akan diisi dengan teks "September".
25.	nama_bulan = "September"	Mengisi variabel nama_bulan with objek string bertuliskan "September".
26.	elif bulan == "10":	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "10". Jika benar, variabel nama_bulan akan diisi dengan teks "Oktober".
27.	nama_bulan = "Oktober"	Mengisi variabel nama_bulan dengan objek string bertuliskan "Oktober".
28.	elif bulan == "11":	Memeriksa kondisi alternatif jika komponen angka bulan bernilai "11". Jika benar, variabel nama_bulan akan diisi dengan teks "November".
29.	nama_bulan = "November"	Mengisi variabel nama_bulan dengan objek string bertuliskan "November".

30.	<code>} else if (bulan == "12") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "12". Jika benar, variabel nama_bulan dialokasikan nilai teks "Desember".
31.	<code>nama_bulan = "Desember"</code>	Mengisi variabel nama_bulan dengan objek string bertuliskan "Desember".
32.	<code>else:</code>	Blok pengecualian akhir yang otomatis dieksekusi jika data pada variabel bulan tidak memenuhi kriteria angka "01" sampai "12" (misal angka 13 atau 00).
33.	<code>nama_bulan = "Bulan tidak valid"</code>	Menugaskan string "Bulan tidak valid" ke dalam variabel nama_bulan sebagai validasi jika masukan kode bulan salah.
34.	<code>print("Tanggal Lahir ASN:", tanggal, nama_bulan, tahun)</code>	[Output] Menggabungkan dan mencetak string konstan beserta isi dari variabel tanggal, nama_bulan, dan tahun secara berurutan ke layar pengguna.
35.	<code>tampilkan_tanggal_lahir ("199301212019031010")</code>	Memanggil (<i>call</i>) fungsi tampilkan_tanggal_lahir dengan mengirimkan argumen string NIP contoh "199301212019031010" untuk dieksekusi oleh program.

b. Penjelasan Tiap Baris Kode R

No	Baris Kode R	Keterangan
1.	<code>tampilkan_tanggal_lahir <- function(nip) {</code>	Mendefinisikan sebuah fungsi bernama tampilkan_tanggal_lahir yang menerima satu argumen/parameter bernama nip. Fungsi ini dibuat agar kode program menjadi lebih modular, rapi, dan mudah dipanggil ulang.
2.	<code>if (nchar(nip) < 8) {</code>	[Validasi Input] Melakukan pengecekan kondisi menggunakan fungsi nchar() untuk menghitung

		jumlah karakter teks variabel nip. Jika panjangnya kurang dari 8 karakter, program tidak akan melanjutkan ekstraksi karena syarat minimal data kelahiran (8 digit pertama) tidak terpenuhi.
3.	<code>cat("NIP tidak valid\n")</code>	Menampilkan pesan kesalahan kepada pengguna di layar konsol R jika jumlah digit NIP yang dikirimkan cacat atau kurang dari 8 karakter, diakhiri dengan karakter <i>newline</i> (\n).
4.	<code>} else {</code>	Menangani alur logika alternatif jika kondisi pada baris ke-2 tidak terpenuhi (artinya panjang teks nip sudah valid, yaitu minimal memiliki 8 karakter atau lebih).
5.	<code>tahun <- substr(nip, 1, 4)</code>	[Proses Slicing] Mengekstrak potongan teks dari karakter posisi ke-1 sampai posisi ke-4 pada variabel nip menggunakan fungsi bawaan <code>substr()</code> untuk mendapatkan komponen tahun lahir ASN, kemudian menyimpannya ke variabel tahun.
6.	<code>bulan <- substr(nip, 5, 6)</code>	[Proses Slicing] Mengekstrak potongan teks dari karakter posisi ke-5 sampai posisi ke-6 pada variabel nip menggunakan fungsi <code>substr()</code> untuk mendapatkan 2 digit komponen angka bulan lahir ASN, kemudian menyimpannya ke variabel bulan.
7.	<code>tanggal <- substr(nip, 7, 8)</code>	[Proses Slicing] Mengekstrak potongan teks dari karakter posisi ke-7 sampai posisi ke-8 pada variabel nip menggunakan fungsi <code>substr()</code> untuk mendapatkan 2 digit komponen angka tanggal lahir ASN, kemudian menyimpannya ke variabel tanggal.
8.	<code>if (bulan == "01") {</code>	[Percabangan Kondisional] Memeriksa apakah isi dari variabel bulan bernilai sama dengan string "01". Jika benar, program akan mengeksekusi blok kode di

		dalam kurung kurawal {} untuk mendefinisikan bulan Januari.
9.	nama_bulan <- "Januari"	Mengisi (menugaskan) variabel nama_bulan dengan nilai teks "Januari" karena kondisi logika pada baris ke-8 terpenuhi.
10.	} else if (bulan == "02") {	(Else If) Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "02". Jika benar, variabel nama_bulan dialokasikan nilai teks "Februari".
11.	nama_bulan <- "Februari"	Mengisi variabel nama_bulan dengan nilai teks "Februari".
12.	} else if (bulan == "03") {	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "03". Jika benar, variabel nama_bulan dialokasikan nilai teks "Maret".
13.	nama_bulan <- "Maret"	Mengisi variabel nama_bulan dengan nilai teks "Maret".
14.	} else if (bulan == "04") {	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "04". Jika benar, variabel nama_bulan dialokasikan nilai teks "April".
15.	nama_bulan <- "April"	Mengisi variabel nama_bulan dengan nilai teks "April".
16.	} else if (bulan == "05") {	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "05". Jika benar, variabel nama_bulan dialokasikan nilai teks "Mei".
17.	nama_bulan <- "Mei"	Mengisi variabel nama_bulan dengan nilai teks "Mei".
18.	} else if (bulan == "06") {	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "06". Jika benar, variabel nama_bulan dialokasikan nilai teks "Juni".
19.	nama_bulan <- "Juni"	Mengisi variabel nama_bulan dengan nilai teks "Juni".

20.	<code>} else if (bulan == "07") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "07". Jika benar, variabel nama_bulan dialokasikan nilai teks "Juli".
21.	<code>nama_bulan <- "Juli"</code>	Mengisi variabel nama_bulan dengan nilai teks "Juli".
22.	<code>} else if (bulan == "08") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "08". Jika benar, variabel nama_bulan dialokasikan nilai teks "Agustus".
23.	<code>nama_bulan <- "Agustus"</code>	Mengisi variabel nama_bulan dengan nilai teks "Agustus".
24.	<code>} else if (bulan == "09") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "09". Jika benar, variabel nama_bulan dialokasikan nilai teks "September".
25.	<code>nama_bulan <- "September"</code>	Mengisi variabel nama_bulan dengan nilai teks "September".
26.	<code>} else if (bulan == "10") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "10". Jika benar, variabel nama_bulan dialokasikan nilai teks "Oktober".
27.	<code>nama_bulan <- "Oktober"</code>	Mengisi variabel nama_bulan dengan nilai teks "Oktober".
28.	<code>} else if (bulan == "11") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "11". Jika benar, variabel nama_bulan dialokasikan nilai teks "November".
29.	<code>nama_bulan <- "November"</code>	Mengisi variabel nama_bulan dengan nilai teks "November".
30.	<code>} else if (bulan == "12") {</code>	Memeriksa kondisi alternatif berikutnya jika variabel bulan bernilai "12". Jika benar, variabel nama_bulan dialokasikan nilai teks "Desember".
31.	<code>nama_bulan <- "Desember"</code>	Mengisi variabel nama_bulan dengan nilai teks "Desember".

32.	<code>} else {</code>	Blok pengecualian akhir (else) yang otomatis tereksekusi apabila masukan string angka komponen bulan pada variabel bulan berada di luar rentang nilai angka "01" hingga "12" (misal angka 15 atau 00).
33.	<code>nama_bulan <- "Bulan tidak valid"</code>	Menugaskan teks "Bulan tidak valid" ke dalam variabel nama_bulan sebagai bentuk deteksi kesalahan data input bulan (<i>error handling</i>).
34.	<code>}</code>	Menutup keseluruhan blok rangkaian dari struktur percabangan kondisional if-else bulan.
35.	<code>cat("Tanggal Lahir ASN:", tanggal, nama_bulan, tahun, "\n")</code>	[Output] Memanggil fungsi cat() untuk menggabungkan string konstan, nilai dari variabel tanggal, nama_bulan, tahun, serta karakter <i>newline</i> (\n) untuk dicetak bersama secara rapi ke layar konsol R.
36.	<code>}</code>	Menutup blok pengecualian (else) dari validasi panjang karakter NIP utama yang dimulai sejak baris ke-4.
37.	<code>}</code>	Menutup tubuh fungsi (function body) dari fungsi tampilkan_tanggal_lahir yang diinisialisasi pada baris ke-1.
38.	<code>tampilkan_tanggal_lahir ("199301212019031010")</code>	Memanggil fungsi tampilkan_tanggal_lahir secara aktif dengan mengirimkan argumen string NIP berupa "199301212019031010" untuk dieksekusi dan diuji di layar konsol R.

5.5 Tabel penjelasan setiap baris kode Python dan R Program 5

a. Penjelasan Tiap Baris Kode R

No	Baris Kode R	Keterangan
1.	<code>A <- c(2, 1, 3)</code>	Membuat objek vektor A yang menyimpan koordinat pusat Cluster A (2, 1, 3) sebagai konstanta. Fungsi <code>c()</code> digunakan untuk menggabungkan elemen menjadi satu kesatuan vektor.
2.	<code>B <- c(1, -4, 6)</code>	Membuat objek vektor B yang menyimpan koordinat pusat Cluster B (1, -4, 6) sebagai konstanta untuk pembandingan jarak nantinya.
3.	<code>C <- c(-2, 3, -2)</code>	Membuat objek vektor C yang menyimpan koordinat pusat Cluster C (-2, 3, -2) sebagai konstanta pembandingan ketiga
4.	<code>x1 <- 1</code>	Menginisialisasi variabel x1 dengan nilai numerik 1 sebagai representasi titik koordinat pada dimensi pertama (Sumbu X).
5.	<code>x2 <- 2</code>	Menginisialisasi variabel x2 dengan nilai numerik 2 sebagai representasi titik koordinat pada dimensi kedua (Sumbu Y).
6.	<code>x3 <- 3</code>	Menginisialisasi variabel x3 dengan nilai numerik 3 sebagai representasi titik koordinat pada dimensi ketiga (Sumbu Z).
7.	<code>U <- c(x1, x2, x3)</code>	Menggabungkan variabel input x1, x2, dan x3 menjadi satu kesatuan objek vektor bernama U agar mempermudah proses manipulasi indeks data.

8.	<code>jarakA <- sqrt((U[1]-A[1])^2 + (U[2]-A[2])^2 + (U[3]-A[3])^2)</code>	Menghitung jarak Euclidean antara titik U dengan pusat A menggunakan rumus Simbol [1], [2], [3] merujuk pada indeks elemen dimensi di dalam vektor R (indeks R dimulai dari 1). Hasilnya disimpan ke variabel jarakA
9.	<code>jarakB <- sqrt((U[1]-B[1])^2 + (U[2]-B[2])^2 + (U[3]-B[3])^2)</code>	Menghitung jarak Euclidean antara titik U dengan pusat B menggunakan struktur formula matematika yang sama, lalu hasilnya disimpan ke variabel jarakB
10.	<code>jarakC <- sqrt((U[1]-C[1])^2 + (U[2]-C[2])^2 + (U[3]-C[3])^2)</code>	Menghitung jarak Euclidean antara titik U dengan pusat C, lalu hasilnya dialokasikan ke dalam variabel jarakC untuk keperluan evaluasi kelompok.
11.	<code>if (jarakA <= jarakB && jarakA <= jarakC) {</code>	Mengevaluasi kondisi logika pertama. Operator && memastikan bahwa jika jarakA terbukti lebih kecil atau sama dengan jarakB DAN jarakC, maka blok kode di dalamnya akan dieksekusi. Penggunaan <= juga berfungsi sebagai penanganan kondisi khusus jika ada jarak yang bernilai sama (<i>tie-breaker</i>).
12.	<code>cluster <- "A"</code>	Mengisi variabel string cluster dengan teks "A", menandakan bahwa titik U secara sah masuk ke dalam kelompok Cluster A.
13.	<code>} else if (jarakB <= jarakA && jarakB <= jarakC) {</code>	Jika kondisi pertama salah, program mengevaluasi alternatif kedua, yaitu menguji apakah jarakB merupakan nilai jarak paling minimal dibandingkan jarakA dan jarakC.

14.	<code>cluster <- "B"</code>	Mengisi variabel string cluster dengan teks "B" jika evaluasi alternatif kedua terpenuhi.
15.	<code>} else {</code>	Blok keputusan akhir (<i>fallback</i>) yang otomatis dieksekusi apabila jarakA maupun jarakB bukan merupakan jarak terpendek.
16.	<code>cluster <- "C"</code>	Mengisi variabel string cluster dengan teks "C", karena secara logis jarakC adalah yang paling minimum.
17.	<code>}</code>	Menandai batas akhir dari seluruh struktur percabangan kondisional (if-else).
18.	<code>cat("Jarak ke A =", jarakA, "\n")</code>	Fungsi cat() digunakan untuk mencetak teks gabungan dan isi variabel jarakA ke layar, diakhiri dengan perintah \n (<i>newline</i>) untuk membuat baris baru.
19.	<code>cat("Jarak ke B =", jarakB, "\n")</code>	Mencetak teks informasi nilai kalkulasi jarakB ke layar agar pengguna dapat melihat transparansi data perhitungan.
20.	<code>cat("Jarak ke C =", jarakC, "\n")</code>	Mencetak teks informasi nilai kalkulasi jarakC ke layar.
21.	<code>cat("Cluster =", cluster)</code>	Menampilkan hasil akhir klasifikasi kelompok cluster (A, B, atau C) yang berhasil didapatkan oleh algoritma.

b. Penjelasan Tiap Baris Kode Python

No	Baris kode Python	Keterangan
1.	<code>import math</code>	Mengimpor modul math untuk menggunakan fungsi matematika seperti sqrt() untuk menghitung akar kuadrat.

2.	$A = (2, 1, 3)$	Membuat tuple A yang berisi koordinat pusat Cluster A yaitu (2,1,3).
3.	$B = (1, -4, 6)$	Membuat tuple B yang berisi koordinat pusat Cluster B yaitu (1,-4,6).
4.	$C = (-2, 3, -2)$	Membuat tuple C yang berisi koordinat pusat Cluster C yaitu (-2,3,-2).
5.	$U = (1, 2, 3)$	Membuat tuple U yang berisi koordinat titik yang akan ditentukan clusternya.
6.	$\text{jarak_A} = \text{math.sqrt}((U[0] - A[0])**2 + (U[1] - A[1])**2 + (U[2] - A[2])**2)$	Menghitung jarak Euclidean antara titik U dan pusat Cluster A menggunakan rumus jarak tiga dimensi.
7.	$\text{jarak_B} = \text{math.sqrt}((U[0] - B[0])**2 + (U[1] - B[1])**2 + (U[2] - B[2])**2)$	Menghitung jarak Euclidean antara titik U dan pusat Cluster B.
8.	$\text{jarak_C} = \text{math.sqrt}((U[0] - C[0])**2 + (U[1] - C[1])**2 + (U[2] - C[2])**2)$	Menghitung jarak Euclidean antara titik U dan pusat Cluster C.
9.	<code>Print("jarak ke A = ", jarak_A</code>	Menampilkan hasil perhitungan jarak antara titik U dan Cluster A.
10.	<code>print("Jarak ke B =", jarak_B)</code>	Menampilkan hasil perhitungan jarak antara titik U dan Cluster B.
11.	<code>print("Jarak ke C =", jarak_C)</code>	Menampilkan hasil perhitungan jarak antara titik U dan Cluster C.
12.	<code>if jarak_A < jarak_B and jarak_A < jarak_C:</code>	Memeriksa apakah jarak ke Cluster A lebih kecil dibandingkan jarak ke Cluster B dan Cluster C.

13.	<code>print("Titik U termasuk Cluster A")</code>	Menampilkan bahwa titik U merupakan anggota Cluster A jika jarak ke Cluster A paling kecil.
14.	<code>elif jarak_B < jarak_A and jarak_B < jarak_C:</code>	Memeriksa apakah jarak ke Cluster B lebih kecil dibandingkan jarak ke Cluster A dan Cluster C.
15.	<code>print("Titik U termasuk Cluster B")</code>	Menampilkan bahwa titik U merupakan anggota Cluster B jika jarak ke Cluster B paling kecil.
16.	<code>else:</code>	Menjalankan perintah berikutnya apabila kondisi Cluster A dan Cluster B tidak terpenuhi.
17.	<code>print("Titik U termasuk Cluster C")</code>	Menampilkan bahwa titik U merupakan anggota Cluster C karena memiliki jarak terdekat atau kondisi sebelumnya tidak terpenuhi.

5.6 Tabel penjelasan setiap baris kode Python dan R Program 6

a. Penjelasan Tiap Baris Kode R

Baris Kode	Penjelasan
<code>p <- 0.6</code>	Menentukan nilai proporsi sampel sebesar 0,6 atau 60%.
<code>n <- 100</code>	Menentukan ukuran sampel sebanyak 100 data.
<code>alpha <- 0.05</code>	Menentukan nilai alpha sebesar 0,05 untuk tingkat kepercayaan 95%.
<code>if (alpha == 0.05)</code>	Mengecek apakah nilai alpha sama dengan 0,05.
<code>z <- 1.96</code>	Menentukan nilai z sebesar 1,96 untuk tingkat kepercayaan 95%.
<code>else if (alpha == 0.10)</code>	Mengecek apakah nilai alpha sama dengan 0,10.
<code>z <- 1.645</code>	Menentukan nilai z sebesar 1,645 untuk tingkat kepercayaan 90%.

<code>else</code>	Menjalankan kondisi jika nilai alpha tidak sesuai.
<code>print("Alpha tidak valid")</code>	Menampilkan pesan kesalahan jika alpha bukan 0,05 atau 0,10.
<code>margin_error <- z * sqrt((p * (1 - p)) / n)</code>	Menghitung margin of error menggunakan rumus interval kepercayaan proporsi.
<code>batas_bawah <- p - margin_error</code>	Menghitung batas bawah interval kepercayaan.
<code>batas_atas <- p + margin_error</code>	Menghitung batas atas interval kepercayaan.
<code>cat("Interval Kepercayaan Proporsi Populasi\n")</code>	Menampilkan judul output program.
<code>cat("-----\n")</code>	Menampilkan garis pemisah agar tampilan output lebih rapi.
<code>cat("Batas bawah =", round(batas_bawah, 4), "\n")</code>	Menampilkan hasil batas bawah dengan 4 angka di belakang koma.
<code>cat("Batas atas =", round(batas_atas, 4), "\n")</code>	Menampilkan hasil batas atas dengan 4 angka di belakang koma.

b. Penjelasan Tiap Baris Kode Python

Baris Kode	Penjelasan
<code>import math</code>	Mengimpor library math untuk menggunakan fungsi matematika seperti akar kuadrat (sqrt).
<code>p = float(input(...))</code>	Meminta pengguna memasukkan nilai proporsi sampel lalu mengubah input menjadi tipe data desimal (float).
<code>n = int(input(...))</code>	Meminta pengguna memasukkan ukuran sampel dan mengubah input menjadi bilangan bulat (integer).
<code>alpha = float(input(...))</code>	Meminta pengguna memasukkan nilai alpha dan mengubahnya menjadi tipe data desimal.
<code>if p < 0 or p > 1:</code>	Mengecek apakah nilai proporsi berada di luar rentang 0 sampai 1.
<code>print("Error: ...")</code>	Menampilkan pesan kesalahan jika nilai proporsi tidak valid.
<code>else:</code>	Menjalankan proses perhitungan jika nilai proporsi valid.
<code>if alpha == 0.10:</code>	Mengecek apakah nilai alpha sama dengan 0,10.
<code>z = 1.645</code>	Menentukan nilai z untuk tingkat kepercayaan 90%.

<code>elif alpha == 0.05:</code>	Mengecek apakah nilai alpha sama dengan 0,05.
<code>z = 1.96</code>	Menentukan nilai z untuk tingkat kepercayaan 95%.
<code>else:</code>	Menjalankan kondisi jika alpha selain 0,10 atau 0,05.
<code>print("Error: Alpha ...")</code>	Menampilkan pesan kesalahan jika alpha tidak valid.
<code>exit()</code>	Menghentikan program jika alpha salah.
<code>margin_error = ...</code>	Menghitung margin of error menggunakan rumus interval kepercayaan proporsi.
<code>batas_bawah = p - margin_error</code>	Menghitung batas bawah interval kepercayaan.
<code>batas_atas = p + margin_error</code>	Menghitung batas atas interval kepercayaan.
<code>print("\nInterval ...")</code>	Menampilkan judul hasil interval kepercayaan.
<code>print("----- ----")</code>	Menampilkan garis pemisah agar output lebih rapi.
<code>print("Batas bawah =", ...)</code>	Menampilkan hasil batas bawah interval kepercayaan.
<code>print("Batas atas =", ...)</code>	Menampilkan hasil batas atas interval kepercayaan.

BAB VI

PENGUJIAN PROGRAM

6.1 Pengujian Program 1

a. Tabel Pengujian

No.	Input	Output yang diharapkan	Output program	Status
1.	Saya bermain basket. Saya juga suka berenang.	Kata = 7, Kalimat = 2	Kata = 7, Kalimat = 2	Berhasil
2.	Halo dunia	Kata = 2, Kalimat = 0	Kata = 2, Kalimat = 0	Berhasil
3.	(kosong)	Kata = 0, Kalimat = 0	Kata = 0, Kalimat = 0	Berhasil

b. Screenshot hasil running

- Skenario 1



```

[1]
✓ 24 d
# Program Menghitung Jumlah Kata dan Jumlah Kalimat

teks = input("Masukkan teks: ")

if teks.strip() == "":
    jumlah_kata = 0
    jumlah_kalimat = 0
else:
    jumlah_kata = len(teks.split())
    jumlah_kalimat = len(teks.split("."))-1

print("\nTeks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")

Masukkan teks: Saya bermain basket. Saya juga suka berenang.

Teks tersebut memuat 2 kalimat dan 7 kata.

```

- Skenario 2

```
[2]
✓ 7d # Program Menghitung Jumlah Kata dan Jumlah Kalimat

teks = input("Masukkan teks: ")

if teks.strip() == "":
    jumlah_kata = 0
    jumlah_kalimat = 0
else:
    jumlah_kata = len(teks.split())
    jumlah_kalimat = len(teks.split("."))-1

print("\nTeks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")

... Masukkan teks: Halo dunia

Teks tersebut memuat 0 kalimat dan 2 kata.
```

- Skenario 3

```
[3]
✓ 1d # Program Menghitung Jumlah Kata dan Jumlah Kalimat

teks = input("Masukkan teks: ")

if teks.strip() == "":
    jumlah_kata = 0
    jumlah_kalimat = 0
else:
    jumlah_kata = len(teks.split())
    jumlah_kalimat = len(teks.split("."))-1

print("\nTeks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")

... Masukkan teks:

Teks tersebut memuat 0 kalimat dan 0 kata.
```

6.3 Pengujian Program 3

a. Tabel Pengujian

No	Input (a,b,c)	Diskriminan	Output
1	1, -5, 6	1	$x_1=3.000$, $x_2=2.000$
2	1, -4, 4	0	$x=2.000$
3	1, 2, 5	-16	Akar-akar imajiner
4	2, -7, 3	25	$x_1=3.000$, $x_2=0.500$
5	1, 6, 9	0	$x=-3.000$

b. Screenshot Hasil Running

- **Skenario 1**

```

def akar_kuadrat(a, b, c):
    D = b**2 - 4*a*c
    if D >= 0:
        x1 = (-b + math.sqrt(D)) / (2*a)
        x2 = (-b - math.sqrt(D)) / (2*a)
        print(f"Akar pertama = {x1:.3f}")
        print(f"Akar kedua = {x2:.3f}")
    elif D < 0:
        x = -b / (2*a)
        print(f"Akar kembar = {x:.3f}")
    else:
        print("Persamaan memiliki akar-akar imajiner")

a = float(input("Masukkan a : "))

```

- **Skenario 2**

```

akar_kuadrat(a,b,c)

Masukkan a : 1
Masukkan b : -5
Masukkan c : 6
Akar pertama = 3.000
Akar kedua = -2.000

```

- **Skenario 3**

```

akar_kuadrat(a,b,c)

Masukkan a : 1
Masukkan b : -4
Masukkan c : 4
Akar kembar = 2.000

```

- **Skenario 4**

```

if D > 0:
    x1 = (-b + math.sqrt(D)) / (2*a)
    x2 = (-b - math.sqrt(D)) / (2*a)
    print("Akar pertama = {x1:.3f}")
    print("Akar kedua = {x2:.3f}")

elif D == 0:
    x = -b / (2*a)
    print("Akar kembar = {x:.3f}")

else:
    print("Persamaan memiliki akar-akar imajiner")

a = float(input("Masukkan a : "))
b = float(input("Masukkan b : "))
c = float(input("Masukkan c : "))
akar_kuadrat(a,b,c)

```

Output: Masukkan a : 1, Masukkan b : 2, Masukkan c : 5, Persamaan memiliki akar-akar imajiner

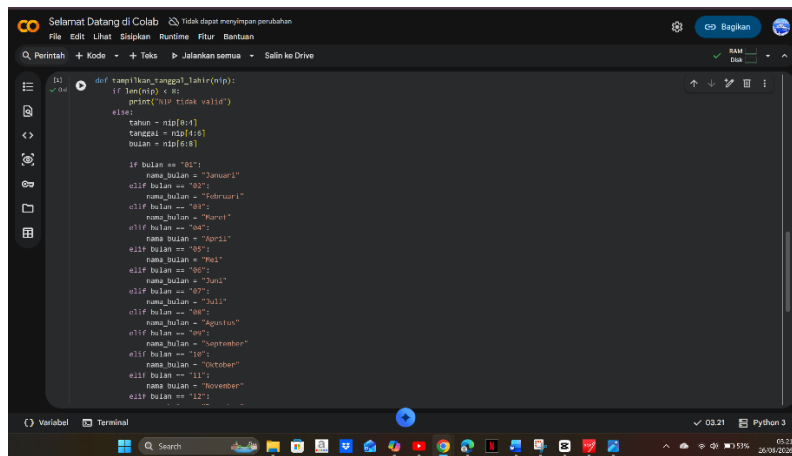
6.4 Pengujian Program 4

a. Tabel Pengujian

No	Jenis Skenario	Input NIP	Output Program
1.	Normal (Valid)	200012312024012002	Tanggal Lahir: 31 Desember 2000
2.	Normal (Valid)	200707052024012002	Tanggal Lahir: 07 Mei 2007
3.	Kondisi Khusus (Digit Kurang)	20070705	Error: NIP harus berjumlah tepat 18 digit.
4.	Kondisi Khusus (Tanggal Tidak Valid)	200705352019031010	Error: Format tanggal tidak valid (01-31).
5.	Kondisi Khusus (Bulan Tidak Valid)	200713052019031010	Error: Format bulan tidak valid (01-12).

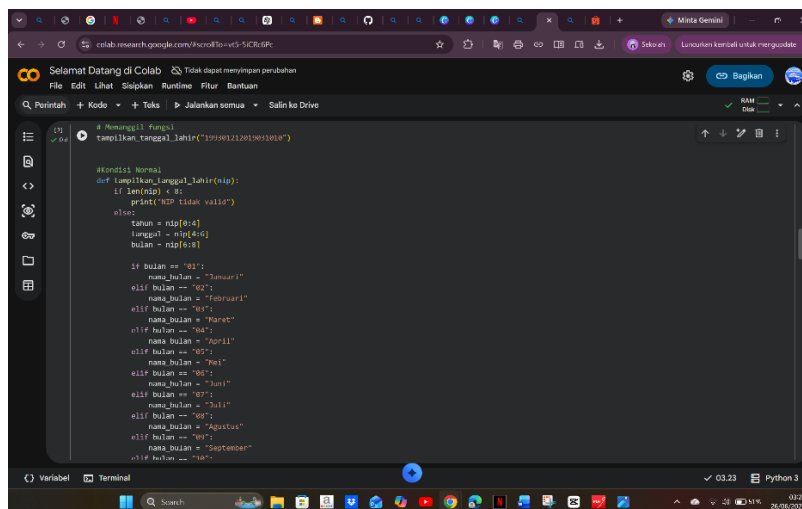
b. Screenshot hasil running

- Skenario 1



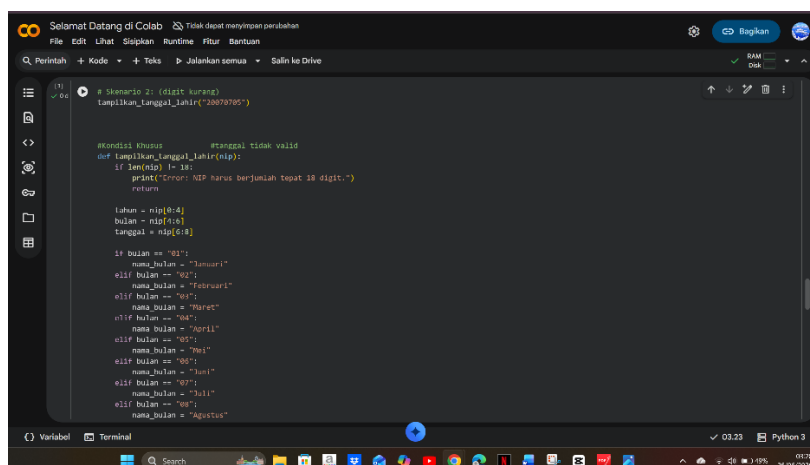
```
def tampilkan_tanggal_lahir(nip):  
    if len(nip) < 8:  
        print("NIP tidak valid")  
    else:  
        tahun = nip[0:4]  
        tanggal = nip[4:6]  
        bulan = nip[6:8]  
  
        if bulan == "01":  
            nama_bulan = "Januari"  
        elif bulan == "02":  
            nama_bulan = "Februari"  
        elif bulan == "03":  
            nama_bulan = "Maret"  
        elif bulan == "04":  
            nama_bulan = "April"  
        elif bulan == "05":  
            nama_bulan = "Mei"  
        elif bulan == "06":  
            nama_bulan = "Juni"  
        elif bulan == "07":  
            nama_bulan = "Juli"  
        elif bulan == "08":  
            nama_bulan = "Agustus"  
        elif bulan == "09":  
            nama_bulan = "September"  
        elif bulan == "10":  
            nama_bulan = "Oktober"  
        elif bulan == "11":  
            nama_bulan = "November"  
        elif bulan == "12":  
            nama_bulan = "Desember"
```

- **Skenario 2**



```
# Kondisi Normal  
def tampilkan_tanggal_lahir(nip):  
    if len(nip) < 8:  
        print("NIP tidak valid")  
    else:  
        tahun = nip[0:4]  
        tanggal = nip[4:6]  
        bulan = nip[6:8]  
  
        if bulan == "01":  
            nama_bulan = "Januari"  
        elif bulan == "02":  
            nama_bulan = "Februari"  
        elif bulan == "03":  
            nama_bulan = "Maret"  
        elif bulan == "04":  
            nama_bulan = "April"  
        elif bulan == "05":  
            nama_bulan = "Mei"  
        elif bulan == "06":  
            nama_bulan = "Juni"  
        elif bulan == "07":  
            nama_bulan = "Juli"  
        elif bulan == "08":  
            nama_bulan = "Agustus"  
        elif bulan == "09":  
            nama_bulan = "September"  
        elif bulan == "10":  
            nama_bulan = "Oktober"  
        elif bulan == "11":  
            nama_bulan = "November"  
        elif bulan == "12":  
            nama_bulan = "Desember"
```

- **Skenario 3**



```
# Skenario 2: (digit kurang)  
tampilkan_tanggal_lahir("20070705")  
  
# Kondisi Khusus # tanggal tidak valid  
def tampilkan_tanggal_lahir(nip):  
    if len(nip) != 18:  
        print("Error: NIP harus berjumlah tepat 18 digit.")  
        return  
  
    tahun = nip[0:4]  
    bulan = nip[4:6]  
    tanggal = nip[6:8]  
  
    if bulan == "01":  
        nama_bulan = "Januari"  
    elif bulan == "02":  
        nama_bulan = "Februari"  
    elif bulan == "03":  
        nama_bulan = "Maret"  
    elif bulan == "04":  
        nama_bulan = "April"  
    elif bulan == "05":  
        nama_bulan = "Mei"  
    elif bulan == "06":  
        nama_bulan = "Juni"  
    elif bulan == "07":  
        nama_bulan = "Juli"  
    elif bulan == "08":  
        nama_bulan = "Agustus"
```

- **Skenario 4**

```

# Scenario 3: (tanggal tidak valid)
tampilkan_tanggal_lahir("20070551207051010")

# Kondisi Khusus
def tampilkan_tanggal_lahir(nip):
    if len(nip) != 18:
        print("Error: NIP harus berjumlah tepat 18 digit.")
        return

    tahun = nip[0:4]
    bulan = nip[4:6]
    tanggal = nip[6:8]

    if bulan == "01":
        nama_bulan = "Januari"
    elif bulan == "02":
        nama_bulan = "Februari"
    elif bulan == "03":
        nama_bulan = "Maret"
    elif bulan == "04":
        nama_bulan = "April"
    elif bulan == "05":
        nama_bulan = "Mei"
    elif bulan == "06":
        nama_bulan = "Juni"
    elif bulan == "07":
        nama_bulan = "Juli"
    elif bulan == "08":
        nama_bulan = "Agustus"

```

- **Skenario 5**

```

elif bulan == "08":
    nama_bulan = "Agustus"
elif bulan == "09":
    nama_bulan = "September"
elif bulan == "10":
    nama_bulan = "Oktober"
elif bulan == "11":
    nama_bulan = "November"
elif bulan == "12":
    nama_bulan = "Desember"
else:
    print("Error: Format bulan tidak valid (01-12).")
    return

if not (tanggal.isdigit() and 1 <= int(tanggal) <= 31):
    print("Error: Format tanggal tidak valid (01-31).")
    return

print("Tanggal Lahir:", tanggal, nama_bulan, tahun)

# Scenario 4: Kondisi Khusus (Bulan Tidak Valid)
tampilkan_tanggal_lahir("20071002207051010")

---
Tanggal Lahir ASN: 21 Januari 1993
Tanggal Lahir ASN: 07 Mei 2007
Error: NIP harus berjumlah tepat 18 digit.
Error: Format tanggal tidak valid (01-31).
Error: Format bulan tidak valid (01-12).

```

6.5 Pengujian Program 5

a. Tabel pengujian

No	Input	Output yang diharapkan	Output Program	Status	Catatan
1.	(7, -2, 4)	Cluster A	Cluster C	Berhasil	Jarak A paling kecil
2.	(2, -5, 7)	Cluster B	Cluster B	Berhasil	Jarak B paling kecil
3.	(-1, 3, -2)	Cluster C	Cluster C	Berhasil	Titik sangat dekat dengan pusat cluster C. Termasuk ke dalam

					kondisi khusus karena titik hampir berimpit dengan pusat Cluster C sehingga menghasilkan jarak yang sangat kecil (1 satuan) dibandingkan cluster lainnya.
--	--	--	--	--	--

1 dan 2 kondisi normal, sedangkan 3 kondisi khusus.

b. Screenshoot hasil running

The first screenshot shows the initial code in the notebook. It imports the math module and defines a function `hitung_jarak` to calculate the Euclidean distance between two points. Below the function, it sets up 'SKENARIO 1 (Normal)' with three points: `(7, -2)`, `(-4, 6)`, and `(-2, -2)`. It then calls the `hitung_jarak` function to calculate the distances between these points.

```
import math

def hitung_jarak(x1, x2, x3, y1, y2, y3):
    return math.sqrt(
        (x1 - y1)**2 +
        (x2 - y2)**2 +
        (x3 - y3)**2
    )

# SKENARIO 1 (Normal)
print("SKENARIO 1")

x1 = 7
x2 = -2
x3 = 4

jarakA = hitung_jarak(x1, x2, x3, 2, 1, 3)
jarakB = hitung_jarak(x1, x2, x3, 1, -4, 6)
jarakC = hitung_jarak(x1, x2, x3, -2, 3, -2)
```

The second screenshot shows the continuation of the code. It implements a clustering logic where the point with the smallest distance to the other two is assigned to a cluster. In this case, since `jarakA` is the smallest, it assigns the point to 'Cluster A'. It then prints the distances and the assigned cluster. Below this, it sets up 'SKENARIO 2 (Normal)' with points `(2, -5)` and `(-5, -5)`.

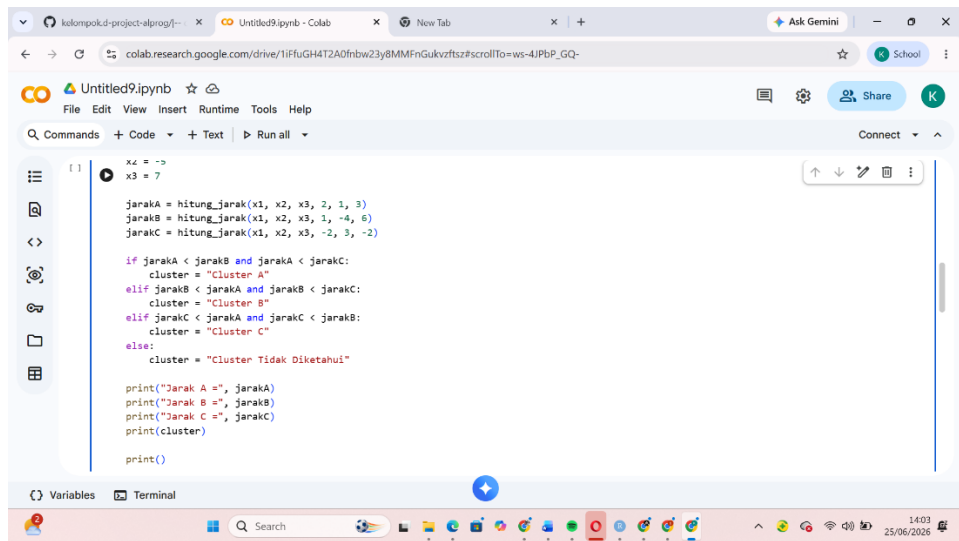
```
if jarakA < jarakB and jarakA < jarakC:
    cluster = "Cluster A"
elif jarakB < jarakA and jarakB < jarakC:
    cluster = "Cluster B"
elif jarakC < jarakA and jarakC < jarakB:
    cluster = "Cluster C"
else:
    cluster = "Cluster Tidak Diketahui"

print("Jarak A =", jarakA)
print("Jarak B =", jarakB)
print("Jarak C =", jarakC)
print(cluster)

print()

# SKENARIO 2 (Normal)
print("SKENARIO 2")

x1 = 2
x2 = -5
```



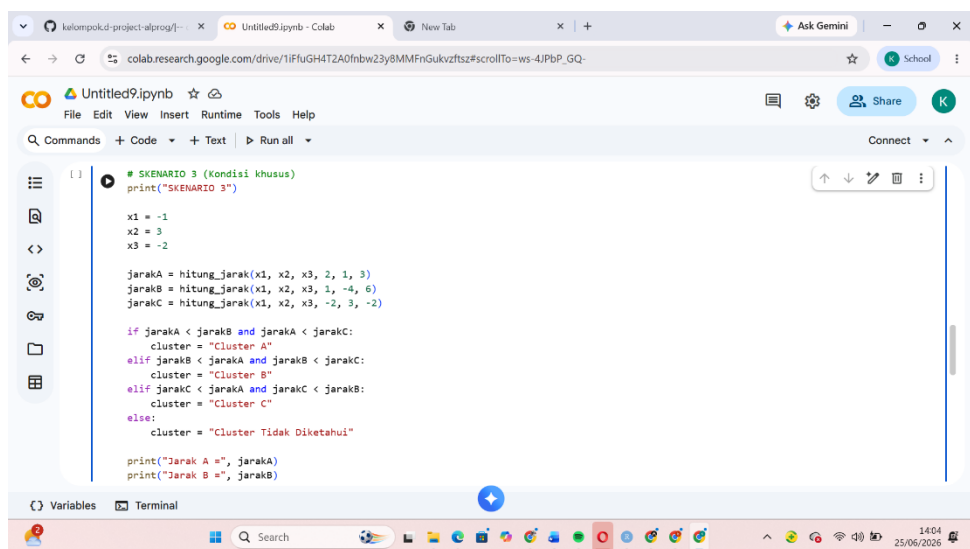
```
x4 = -3
x3 = 7

jarakA = hitung_jarak(x1, x2, x3, 2, 1, 3)
jarakB = hitung_jarak(x1, x2, x3, 1, -4, 6)
jarakC = hitung_jarak(x1, x2, x3, -2, 5, -2)

if jarakA < jarakB and jarakA < jarakC:
    cluster = "Cluster A"
elif jarakB < jarakA and jarakB < jarakC:
    cluster = "Cluster B"
elif jarakC < jarakA and jarakC < jarakB:
    cluster = "Cluster C"
else:
    cluster = "Cluster Tidak Diketahui"

print("Jarak A =", jarakA)
print("Jarak B =", jarakB)
print("Jarak C =", jarakC)
print(cluster)

print()
```



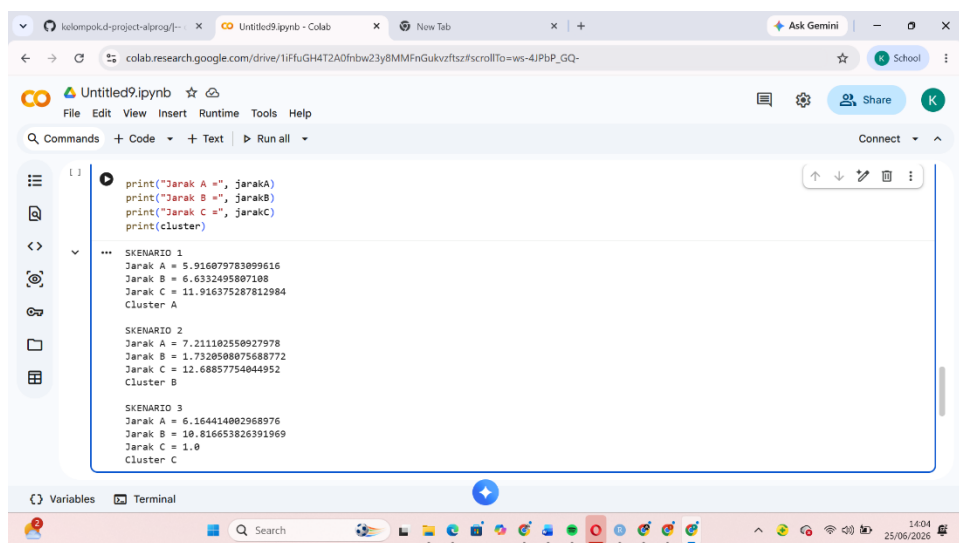
```
# SKENARIO 3 (Kondisi khusus)
print("SKENARIO 3")

x1 = -1
x2 = 3
x3 = -2

jarakA = hitung_jarak(x1, x2, x3, 2, 1, 3)
jarakB = hitung_jarak(x1, x2, x3, 1, -4, 6)
jarakC = hitung_jarak(x1, x2, x3, -2, 5, -2)

if jarakA < jarakB and jarakA < jarakC:
    cluster = "Cluster A"
elif jarakB < jarakA and jarakB < jarakC:
    cluster = "Cluster B"
elif jarakC < jarakA and jarakC < jarakB:
    cluster = "Cluster C"
else:
    cluster = "Cluster Tidak Diketahui"

print("Jarak A =", jarakA)
print("Jarak B =", jarakB)
```



```
print("Jarak A =", jarakA)
print("Jarak B =", jarakB)
print("Jarak C =", jarakC)
print(cluster)

SKENARIO 1
Jarak A = 5.916079783099616
Jarak B = 6.6332495807108
Jarak C = 11.916375287812984
Cluster A

SKENARIO 2
Jarak A = 7.211102550927978
Jarak B = 1.7320508075688772
Jarak C = 12.68857754044952
Cluster B

SKENARIO 3
Jarak A = 6.164414002960976
Jarak B = 10.816653826391969
Jarak C = 1.0
Cluster C
```

c. Interpretasi

Berdasarkan hasil pengujian, program berhasil menentukan cluster terdekat untuk setiap titik yang diuji. Pada skenario pertama titik U diklasifikasikan ke Cluster A karena memiliki jarak paling kecil terhadap pusat Cluster A. Pada skenario kedua titik U diklasifikasikan ke Cluster B karena jaraknya paling dekat dengan pusat Cluster B. Sementara itu, pada skenario ketiga titik U diklasifikasikan ke Cluster C karena memiliki jarak paling kecil terhadap pusat Cluster C. Skenario ketiga termasuk kondisi khusus karena titik U berada sangat dekat dengan pusat Cluster C, sehingga menghasilkan jarak yang jauh lebih kecil dibandingkan jarak ke cluster lainnya. Seluruh hasil pengujian sesuai dengan output yang diharapkan, sehingga program dinyatakan mampu melakukan klasifikasi cluster dengan benar menggunakan metode jarak Euclidean.

6.6 Pengujian Program 6

a. Tabel Pengujian

No	Input	Alpha	Hasil	Keterangan
1	p=0.6, n=100	0.05	Interval berhasil dihitung	Valid
2	p=0.4, n=50	0.10	Interval berhasil dihitung	Valid
3	p=1.5, n=80	0.05	Muncul pesan error	Tidak valid

b. Screenshot hasil running

• Skenario 1

```

import math

# Input data
n = float(input("Masukkan proporsi sampel (n): "))
p = float(input("Masukkan ukuran sampel (p): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi nilai proporsi
if p < 0 or p > 1:
    print("Error: Nilai proporsi harus antara 0 dan 1")
else:
    # Menghitung nilai z berdasarkan alpha
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96
    else:
        print("Error: alpha harus 0.10 atau 0.05")
        exit()

    # Menghitung margin of error
    margin_error = z * math.sqrt(p * (1 - p) / n)

    # Menghitung batas bawah dan batas atas
    batas_bawah = p - margin_error
    batas_atas = p + margin_error

    # Menampilkan hasil
    print("Interval Kepercayaan Proporsi Populasi")
    print("-----")
    print("Batas bawah: ", round(batas_bawah, 4))
    print("Batas atas: ", round(batas_atas, 4))

# Menampilkan hasil
print("Masukkan proporsi sampel (n): 100")
print("Masukkan ukuran sampel (p): 0.6")
print("Masukkan nilai alpha (0.10 atau 0.05): 0.05")
print("Interval Kepercayaan Proporsi Populasi")
print("-----")
print("Batas bawah: 0.4")

```

• Skenario 2

```
# Input data
n = float(input("Masukkan proporsi sampel (p): "))
p = float(input("Masukkan standar sampel (q): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi nilai proporsi
if p < 0 or p > 1:
    print("Error: Nilai proporsi harus antara 0 dan 1")
else:
    # Menentukan nilai z berdasarkan alpha
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96
    else:
        print("Error: Alpha harus 0.10 atau 0.05")
        exit()

# Menghitung margin of error
margin_error = z * math.sqrt(p * (1 - p)) / n

# Menghitung batas bawah dan batas atas
batas_bawah = p - margin_error
batas_atas = p + margin_error

# Menampilkan hasil
print("Interval Kepercayaan Proporsi Populasi")
print("-----")
print("Batas bawah: ", round(batas_bawah, 4))
print("Batas atas: ", round(batas_atas, 4))

# Hasil program
print("Masukkan proporsi sampel (p): 0.4")
print("Masukkan standar sampel (q): 0.4")
print("Masukkan nilai alpha (0.10 atau 0.05): 0.10")
print("Interval Kepercayaan Proporsi Populasi")
print("Batas bawah: 0.286")
print("Batas atas: 0.514")
```

• **Skenario 3**

```
# Input data
n = float(input("Masukkan proporsi sampel (p): "))
p = float(input("Masukkan standar sampel (q): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi nilai proporsi
if p < 0 or p > 1:
    print("Error: Nilai proporsi harus antara 0 dan 1")
else:
    # Menentukan nilai z berdasarkan alpha
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96
    else:
        print("Error: Alpha harus 0.10 atau 0.05")
        exit()

# Menghitung margin of error
margin_error = z * math.sqrt(p * (1 - p)) / n

# Menghitung batas bawah dan batas atas
batas_bawah = p - margin_error
batas_atas = p + margin_error

# Menampilkan hasil
print("Interval Kepercayaan Proporsi Populasi")
print("-----")
print("Batas bawah: ", round(batas_bawah, 4))
print("Batas atas: ", round(batas_atas, 4))

# Hasil program
print("Masukkan proporsi sampel (p): 0.5")
print("Masukkan standar sampel (q): 0.5")
print("Masukkan nilai alpha (0.10 atau 0.05): 0.05")
print("Error: Nilai proporsi harus antara 0 dan 1")
```

BAB VII

PENUTUP

7.1 Kesimpulan

Berdasarkan hasil pengerjaan project ini, kelompok kami berhasil menyelesaikan keenam program menggunakan Python dan R, yaitu analisis teks, manipulasi string, akar persamaan kuadrat, pengolahan NIP ASN, klasifikasi cluster 3 dimensi, dan interval konfidensi proporsi. Selama proses pengerjaan, kami belajar bahwa membuat pseudocode dan flowchart terlebih dahulu sangat membantu dalam memahami alur logika program sebelum mulai menulis kode, dan meskipun Python dan R memiliki cara penulisan yang berbeda, logika yang digunakan tetap sama sehingga memahami algoritmanya jauh lebih penting daripada sekadar menghafal sintaks. Kami juga memahami pentingnya penggunaan fungsi dalam program, terutama pada Program 5 dan Program 6, karena membuat kode menjadi lebih rapi dan lebih mudah diperbaiki jika ada kesalahan. Selain itu, dari proses pengujian kami belajar bahwa program yang baik harus bisa menangani berbagai kemungkinan input, termasuk input yang salah atau tidak valid, sehingga program tidak mudah error saat digunakan oleh orang lain. Secara keseluruhan, project ini memberikan pengalaman yang sangat bermanfaat bagi kami dalam menerapkan konsep algoritma dan pemrograman ke dalam kasus nyata, sekaligus melatih kemampuan kerja sama tim melalui penggunaan GitHub.

7.2 Kendala

Selama proses pengerjaan project ini, terdapat beberapa kendala yang kami hadapi. Kendala pertama adalah penggunaan GitHub, karena ini merupakan pertama kalinya kami menggunakan platform tersebut sehingga kami perlu waktu untuk beradaptasi dan mempelajarinya terlebih dahulu. Meskipun begitu, kami akhirnya bisa menggunakannya dengan baik setelah belajar bersama dan saling membantu sesama anggota. Kendala lainnya adalah menyamakan jadwal seluruh anggota kelompok untuk mengerjakan project bersama, karena masing-masing anggota memiliki kesibukan masing-masing

sehingga kami perlu menyesuaikan waktu dengan baik agar pengerjaan project tetap berjalan lancar dan selesai tepat waktu.

7.3 Saran

Saran pengembangan program ke depannya, program yang sudah dibuat bisa dikembangkan lagi menjadi lebih baik. Misalnya dengan menambahkan tampilan yang lebih menarik agar lebih mudah digunakan, menambahkan fitur-fitur baru yang lebih lengkap, serta memperbaiki program agar bisa menangani lebih banyak kemungkinan input dari pengguna. Dengan begitu, program yang dibuat bisa menjadi lebih bermanfaat dan lebih mudah digunakan oleh siapa saja.

LAMPIRAN

1. Link GitHub

<https://github.com/nadienanggraeni-art/kelompok.d-project-alprog.git>

2. Link Vidio Demo

<https://drive.google.com/file/d/14oiXZcfQ9KC4pwXRIYzO8aNLOLPObZ1o/view?usp=sharing>

3. Link Penggunaan AI

<https://chatgpt.com/share/6a3e959b-952c-83ec-b846-5286e1cb85df>

<https://chatgpt.com/share/6a3e8222-c788-83ec-87a4-f5af31e6eb05>

<https://chatgpt.com/share/6a3e80b8-6bd8-83ec-8b38-f9dd84d87bb7>

<https://gemini.google.com/share/d/1o40gSDwSf2JI3yAAo4qAIYqXHrhj-nAU?usp=sharing>

<https://chatgpt.com/c/6a301bb6-f280-83ec-af52-a1833a17124f>